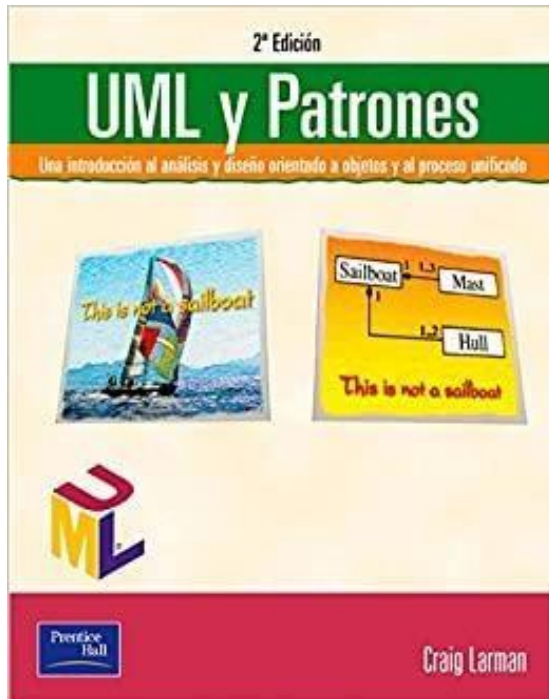


Algunos principios básicos de diseño orientado a objetos

Contenidos

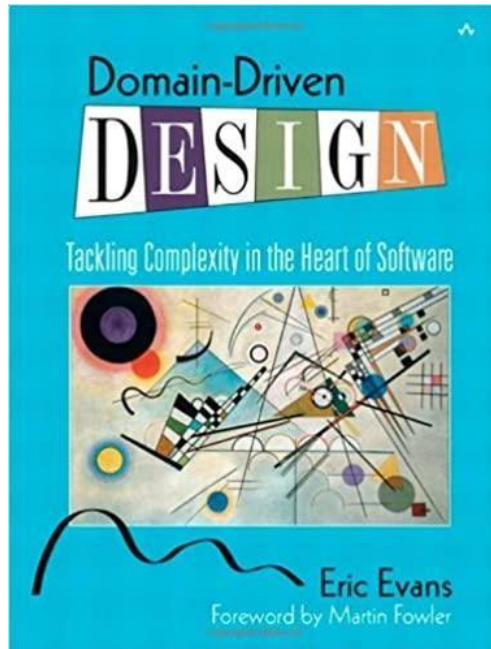
- Patrones GRASP
 - Experto, Creador y Controlador
 - Principio *Separación Modelo-Vista*
- Principios básicos OO
 - Favorecer el cambio
 - El rol de las interfaces y polimorfismo (ya conocido)
 - Ocultación de Información (ya conocido)
- Desarrollo dirigido por el dominio (DDD)

Principal Referencia Bibliográfica



Craig Larman,
*“UML y Patrones: Una introducción
al análisis y diseño orientado a
objetos y al proceso unificado”,*
Prentice-Hall, 2003

Bibliografía DDD



Eric Evans, “*Domain-Driven Design*”, Addison-Wesley, 2004.
(sólo para conocer la principal fuente)

- Avram, F. Marinescu. “Domain-Driven Design Quickly”, InfoQ, 2006. Resumen del libro “Domain-Driven Design” de Eric-Evans, (<http://www.infoq.com/minibooks/domain-driven-design-quickly>)
- “DDD: Principios, beneficios y elementos”, Jonathan Loscalzo, abril, 2018, Medium (<https://bit.ly/3ZwDDFN>)
- Por supuesto, ChatGPT 😊

Patrones GRASP

- “Describen los principios básicos de asignación de responsabilidades a clases”.
- “Distribuir responsabilidades en la parte más difícil del diseño OO. Consume la mayor parte del tiempo”.
- Patrones GRASP:
 - Experto, Creador, Controlador***
 - Bajo Acoplamiento y Alta Cohesión*
 - Polimorfismo y Proteger variación*

Responsabilidades y Métodos

- Una **responsabilidad se implementa mediante uno o más métodos** y puede asignarse a varias clases según la granularidad (tamaño).
- “Contrato u obligación de una clase”. Dos categorías:
 - **Conocer**
 - datos encapsulados privados
 - existencia de objetos conectados
 - datos derivados o calculados
 - **Hacer**
 - Algo él mismo, como crear un objeto o realizar un cálculo
 - Iniciar una acción en otros objetos
 - Controlar y coordinar actividades en otros objetos
- **Diagramas de interacción** representan una asignación de responsabilidades.

Responsabilidades y Métodos

```
public class Cuenta {  
    private int saldo;  
    private Cliente cliente;  
    private CodigoCuenta ccc;  
    ..  
}
```

Conocer cliente, saldo, código,...

```
    public int reintegro(int cantidad) {...}  
    public boolean comprobarOk() {...}  
    private void registrarTransaccion() {...}  
    ..
```

Hacer un reintegro
(responsabilidad)

Y pueden involucrar
métodos de otras clases

Experto

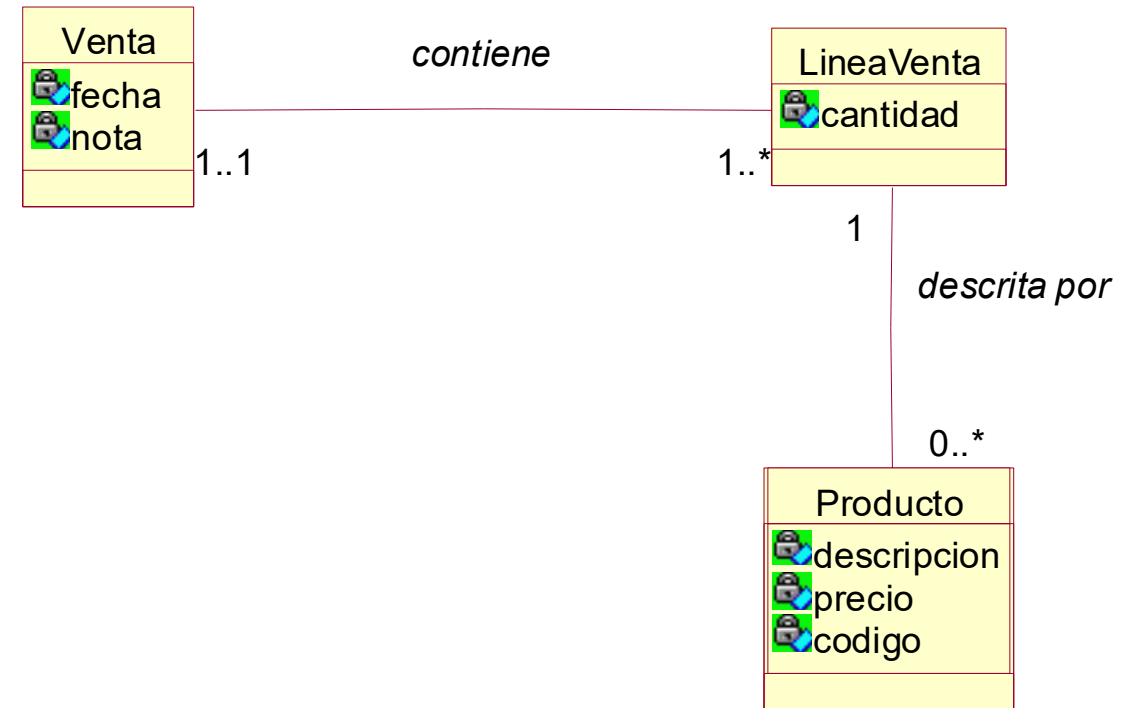
¿Cómo se asignan responsabilidades?

“Asignar una responsabilidad a la clase que tiene la información necesaria para cumplimentarla”

- Heurísticas relacionadas
 - Distribuir responsabilidades de forma homogénea
 - No crear clases “dios”
- Beneficios
 - Se conserva encapsulación: Bajo acoplamiento
 - Alta Cohesión: clases más ligeras

Ejemplo

¿Quién es el responsable de conocer el total de una venta?

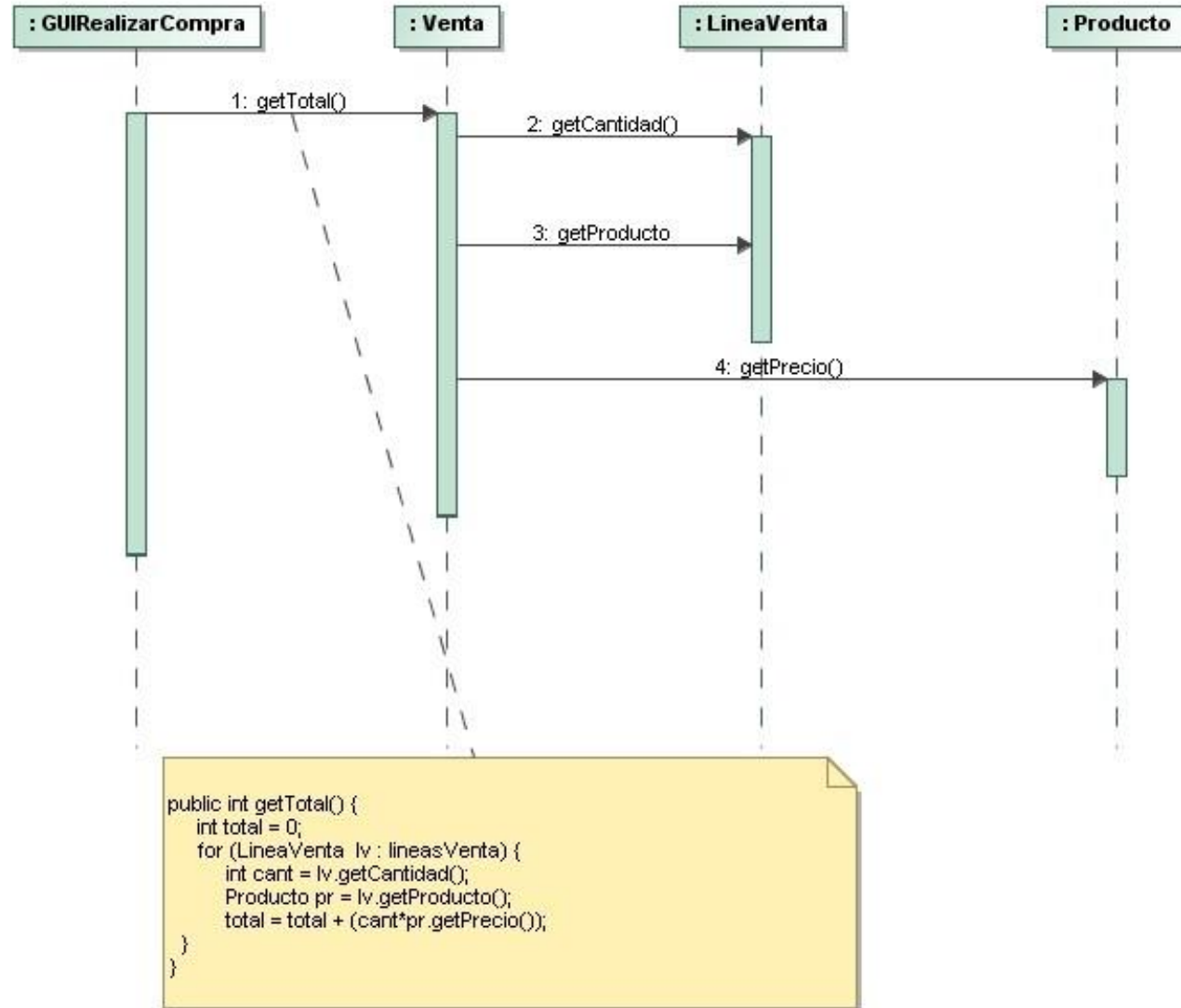


Ejemplo 1

```
public double getTotal() {  
    double total = 0.0;  
    for (LineaVenta lv : lineasVenta) {  
        double cant = lv.getCantidad();  
        Producto pr = lv.getProducto();  
        total = total + (cant*pr.getPrecio());  
    }  
    return total;  
}
```

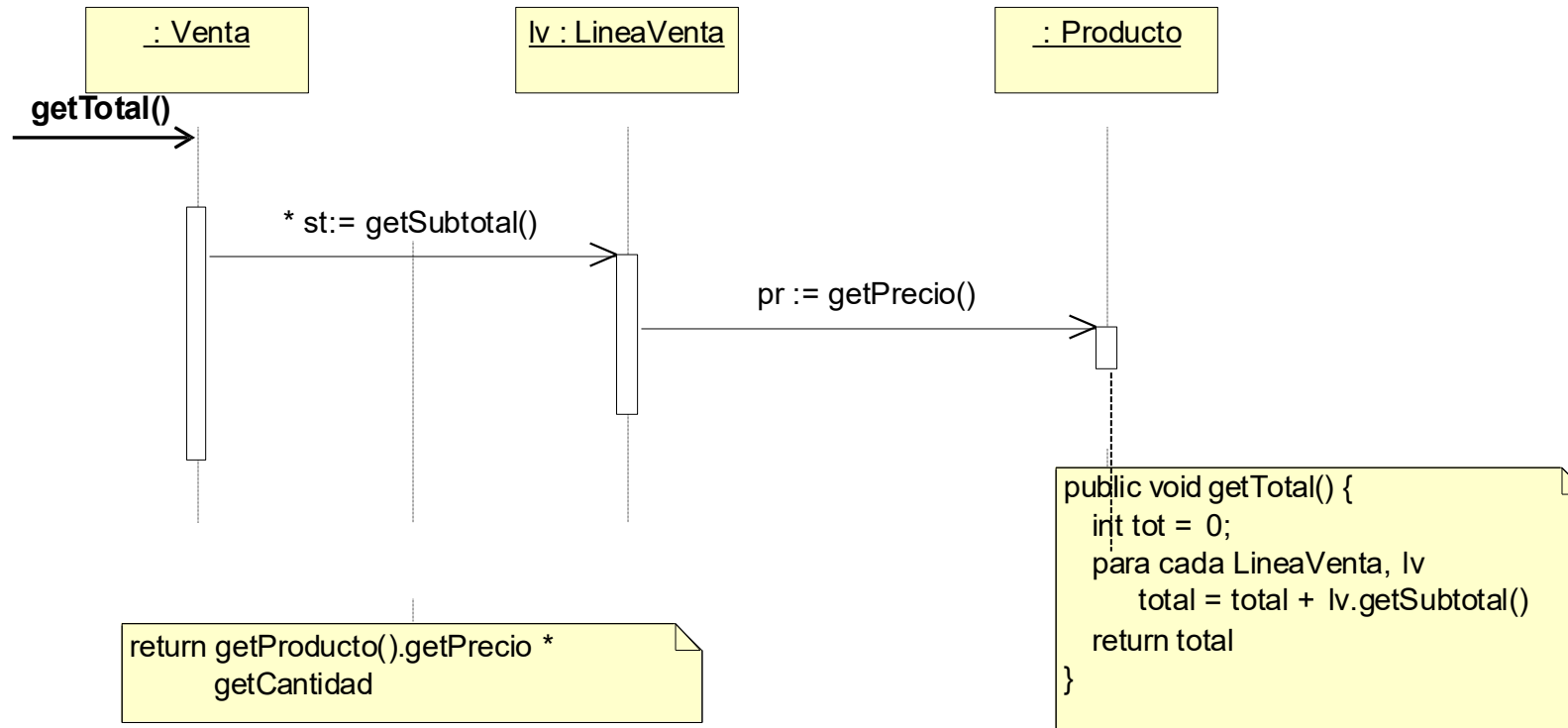
¿Escribimos este código?

Ejemplo 1. Con violación del experto



Ejemplo 1. Sin violar Experto

¿Quién es el responsable de conocer el total de una venta?

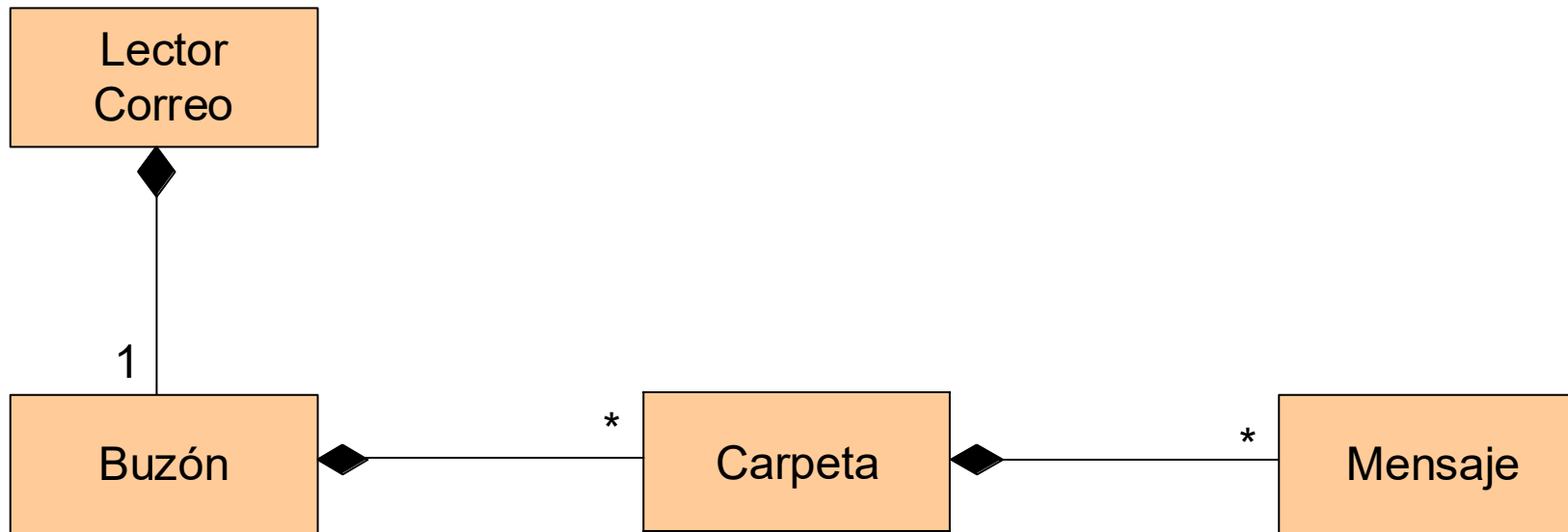


Ejemplo 1. Sin violar Experto

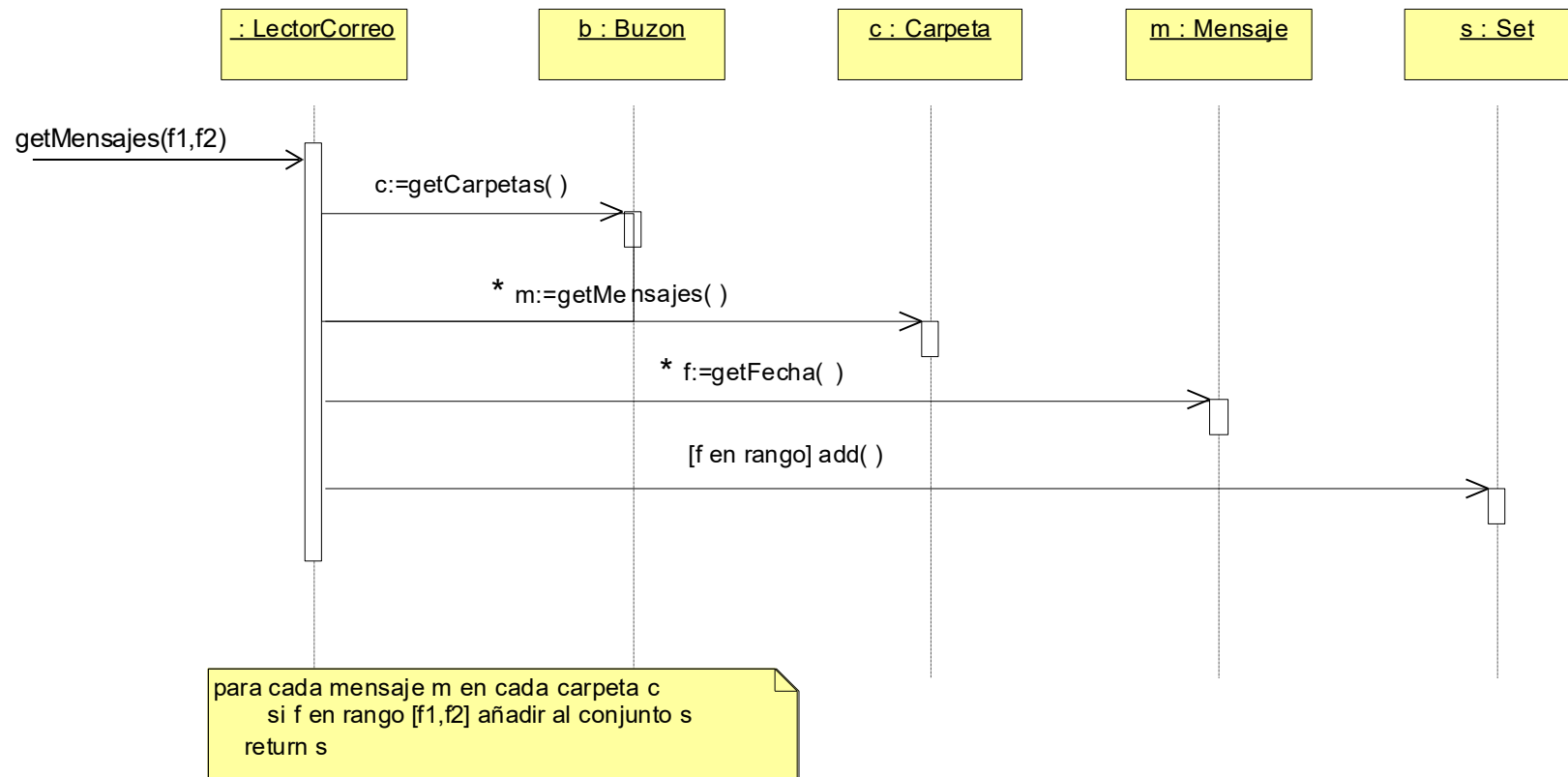
```
class Venta {..  
    public double getTotal() {  
        double total = 0.0;  
        for (LineaVenta lv : lineasVenta) {  
            total = total + lv.getSubtotal(); }  
        return total;  
    }  
}  
  
class LineaVenta{..  
    private int cantidad;  
    private Producto producto;  
    public double getSubtotal() {  
        return cantidad*producto.getPrecio() }  
}
```

Ejemplo 2: Lector Emails

¿Quién es el responsable de conocer todos los mensajes recibidos entre dos fechas en un buzón de emails?

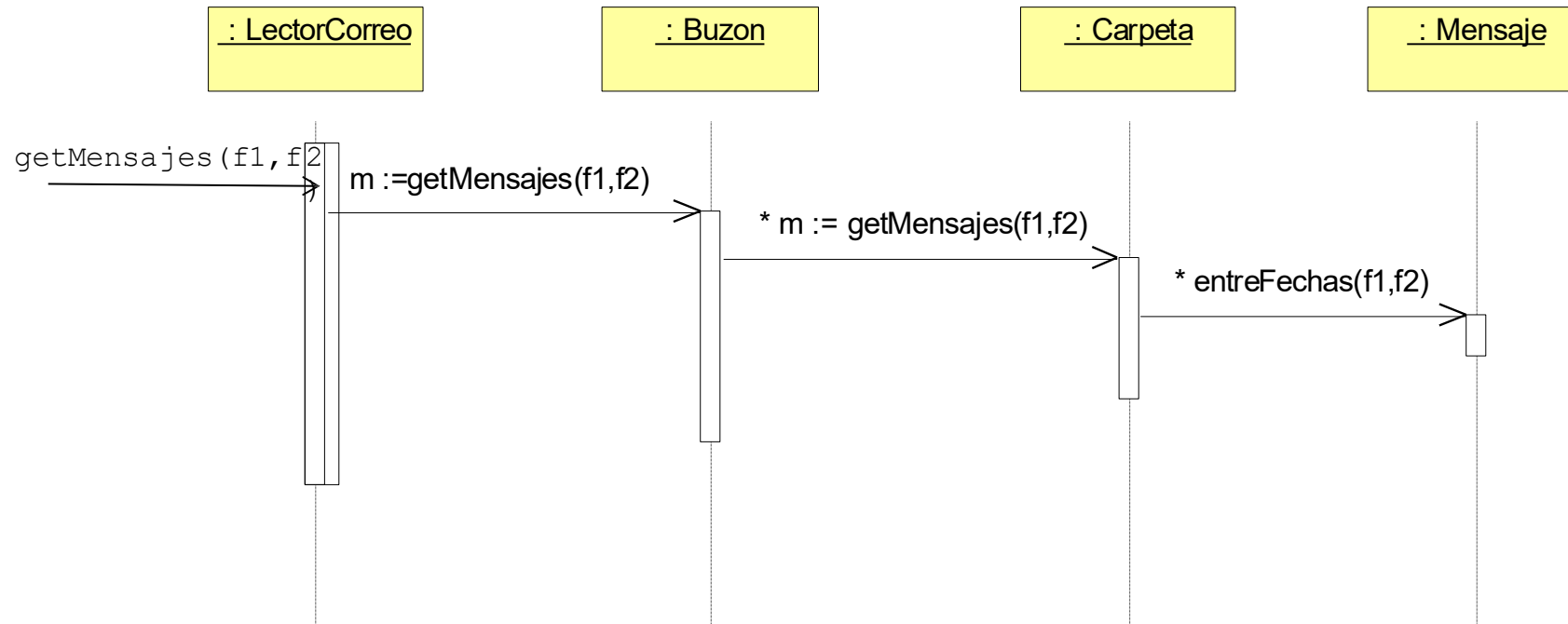


Ejemplo 2. Con violación del “Experto”



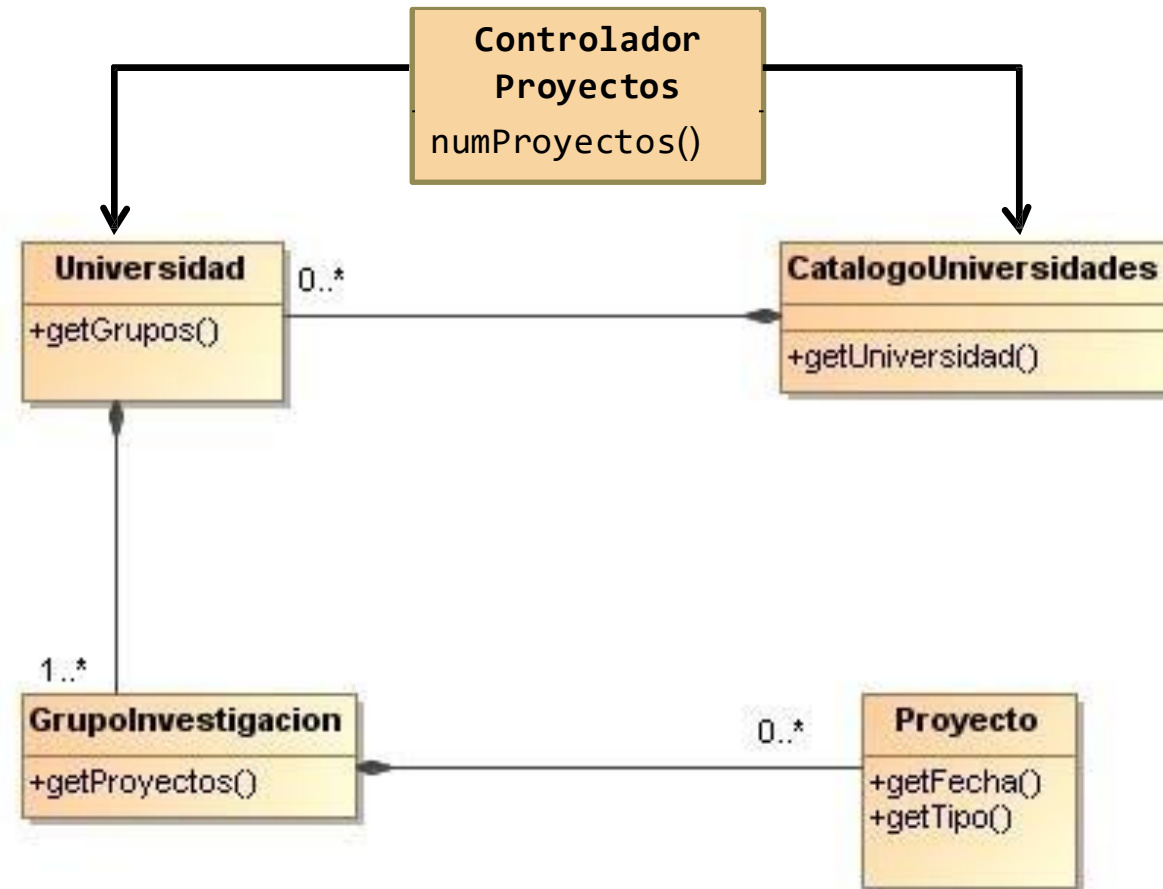
Ejemplo 2. Sin violar Experto

¿Quién es el responsable de conocer todos los mensajes recibidos entre dos fechas?



Ejemplo 3: Indicadores universidades

¿Quién es el responsable de conocer el **número de proyectos de cierto tipo** realizados en una universidad en un rango de fechas ?

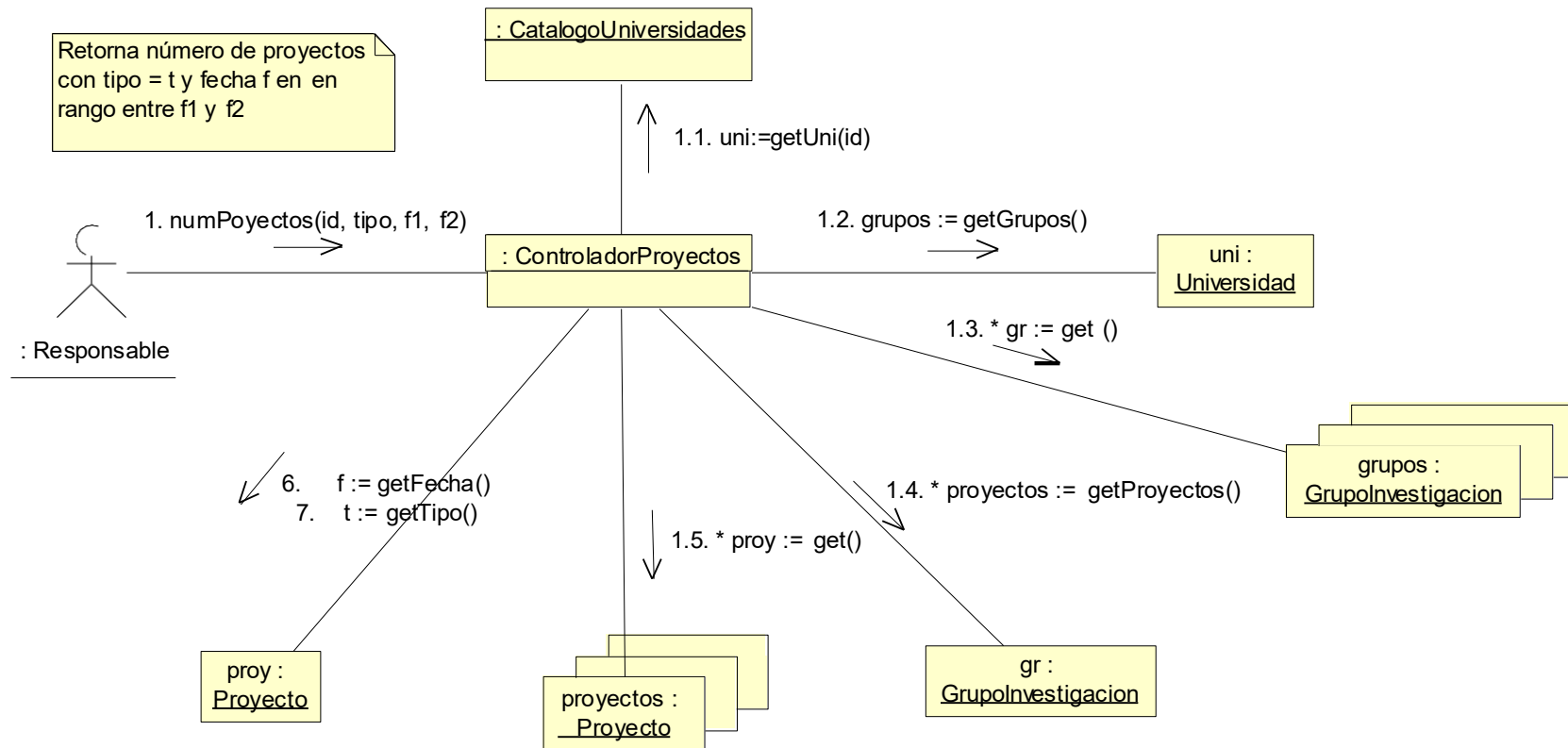


Ejemplo 3. Con violación del “experto”

```
public class ControladorProyectos {...  
    public int numProyectos(Universidad uni, int tipo, Date f1, Date f2){  
        int totalProyectos = 0;  
        Collection<Grupo> grupos = uni.getGrupos();  
        for (Grupo grupo: grupos) {  
            Collection<Proyecto> proyectos = grupo.getProyectos();  
            for (Proyecto proyecto : proyectos){  
                fechaProyecto = proyecto.getFecha();  
                if((fechaProyecto.before(f2))  
                    &&(fechaProyecto.after(f1))  
                    && (tipo == proyecto.getTipo()))  
                    totalProyectos= totalProyectos + 1;  
            }  
        }  
        return totalProyectos;  
    }  
}
```

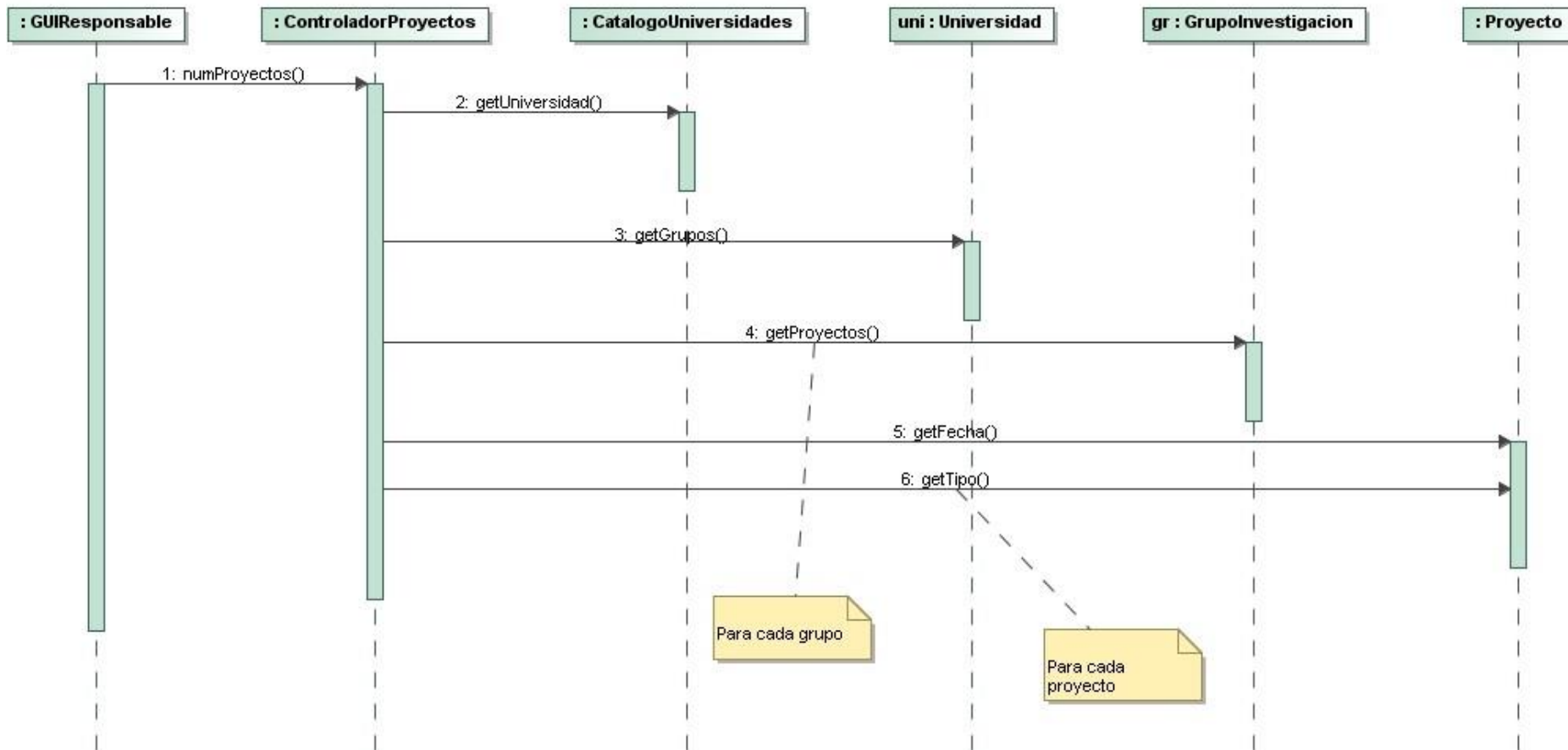
Ejemplo 3. Con violación del “Experto”

¿Quién es el responsable de conocer el **número de proyectos de cierto tipo** realizados en una universidad en un rango de fechas ?



Ejemplo 3. Con violación del “Experto”

¿Quién es el responsable de conocer el **número de proyectos de cierto tipo** realizados en una universidad en un rango de fechas ?



Ejemplo 3. Sin violar Experto

```
public class ControladorProyectos {..  
    public int numProyectos(Universidad uni, int tipo, Date f1, Date f2)  
    {  
        return uni.numProyectos(tipo, f1, f2);  
    }  
}
```

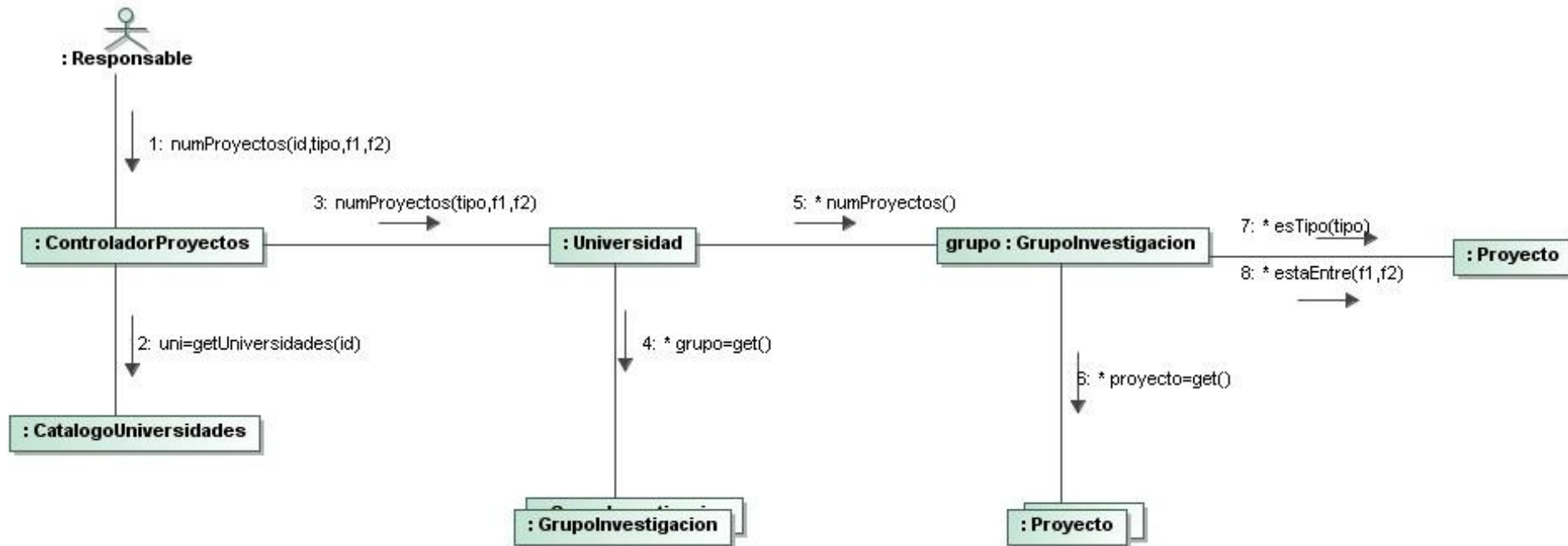
```
public class Universidad {...  
    private Collection<GrupoInvestigacion> grupos;  
    public int numProyectos(int tipo, Date f1, Date f2) {  
        int total = 0;  
        for (Grupo grupo: grupos)  
            total = total + grupo.numProyectos(tipo, f1, f2);  
        return total;  
    }  
}
```

Ejemplo 3. Sin violar Experto

```
public class GrupoInvestigacion {...  
    private List<Proyecto> proyectos;  
    public int numProyectos(int tipo, Date f1, Date f2) {  
        int total = 0;  
        for (Proyecto proyecto : proyectos){  
            if ((proyecto.esTipo(tipo)) &&  
                (proyecto.estaEntre(f1, f2)))  
                total = total + 1;  
        }  
        return total;  
    }  
}  
  
public class Proyecto {...  
    private Date fecha;  
    private int tipo;  
    public boolean esTipo(int tipo) {  
        return (this.tipo == tipo);  
    }  
    public boolean estaEntre(Date f1, Date f2) {  
        return ((fecha.after(f1)) && (fecha.before(f2)));  
    }  
}
```

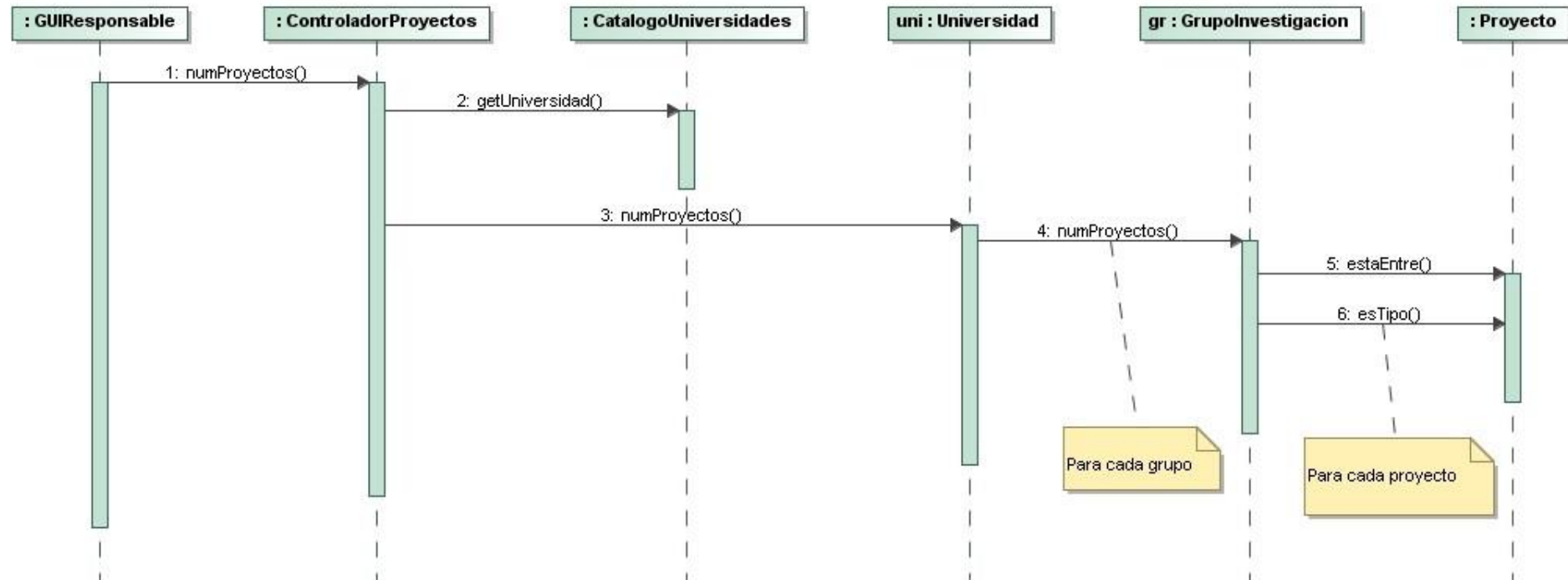
Ejemplo 3. Sin violar Experto

¿Quién es el responsable de conocer el **número de proyectos de cierto tipo** realizados en una universidad en un rango de fechas ?



Ejemplo 3. Sin violar Experto

¿Quién es el responsable de conocer el **número de proyectos de cierto tipo** realizados en una universidad en un rango de fechas ?



Creador

¿Quién es responsable de crear una nueva instancia de una cierta clase?

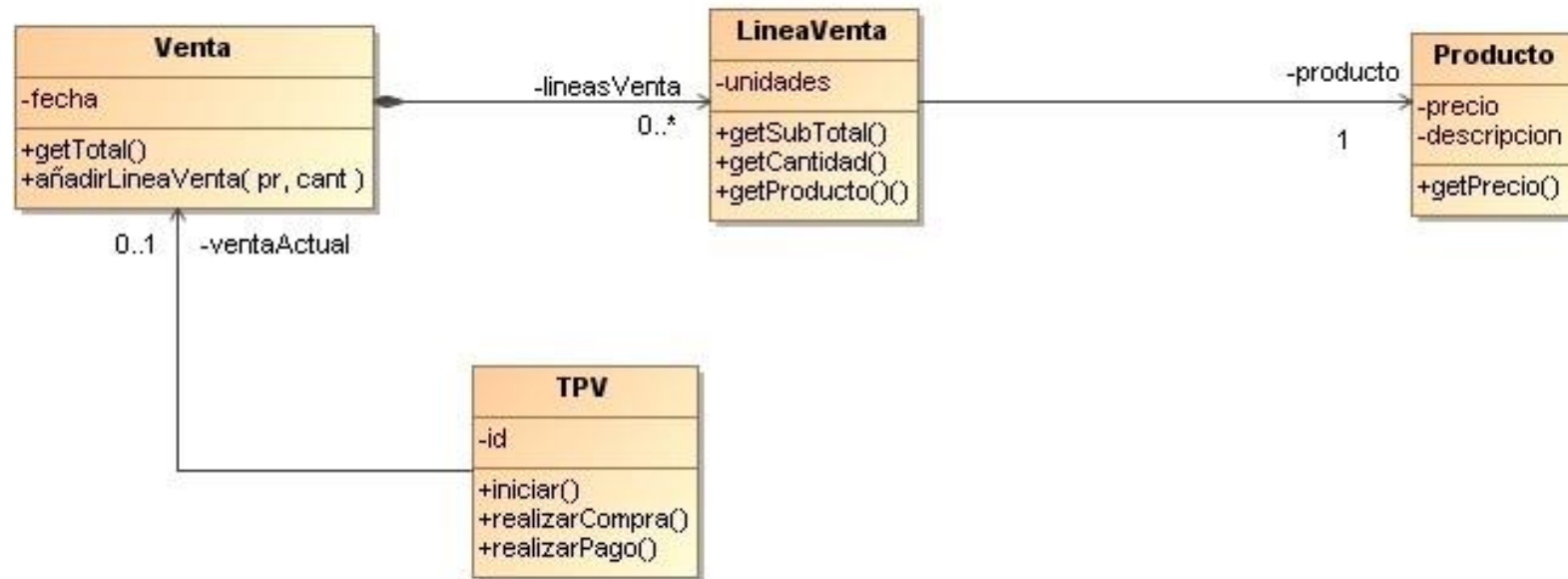
“Asignar a la clase B la responsabilidad de crear instancias de una clase A si:

- B es una **agregación** de objetos de A
- B **contiene** objetos de A
- B **registra** instancias de A
- B **hace un uso específico** de los objetos de A
- B **proporciona los datos de inicialización** necesarios para crear un objeto de A”

- Beneficios:
 - Bajo acoplamiento

Ejemplo 1

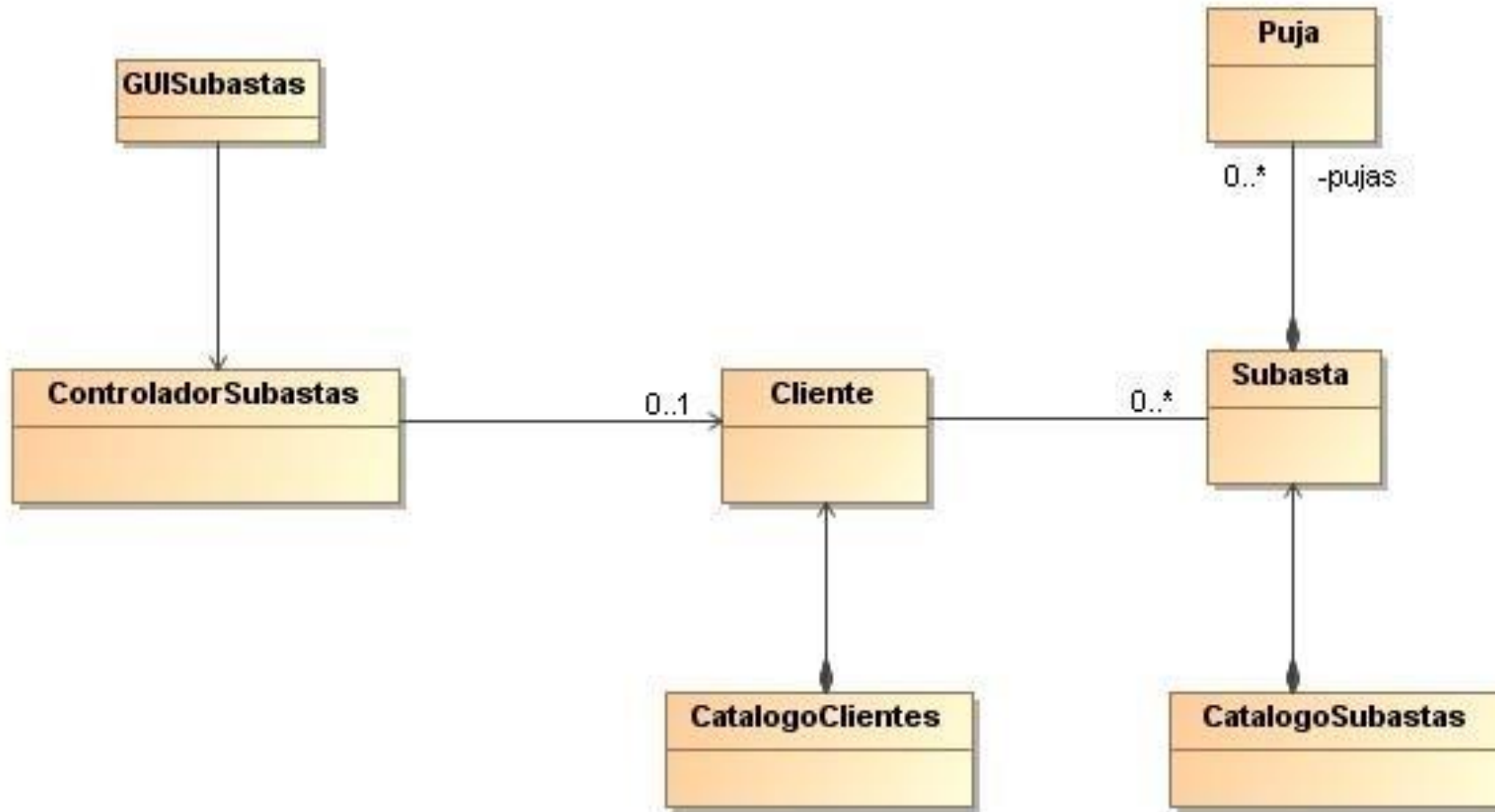
- Venta debería crear las instancias de LineaVenta



- ¿Quién debería crear instancias de Venta?

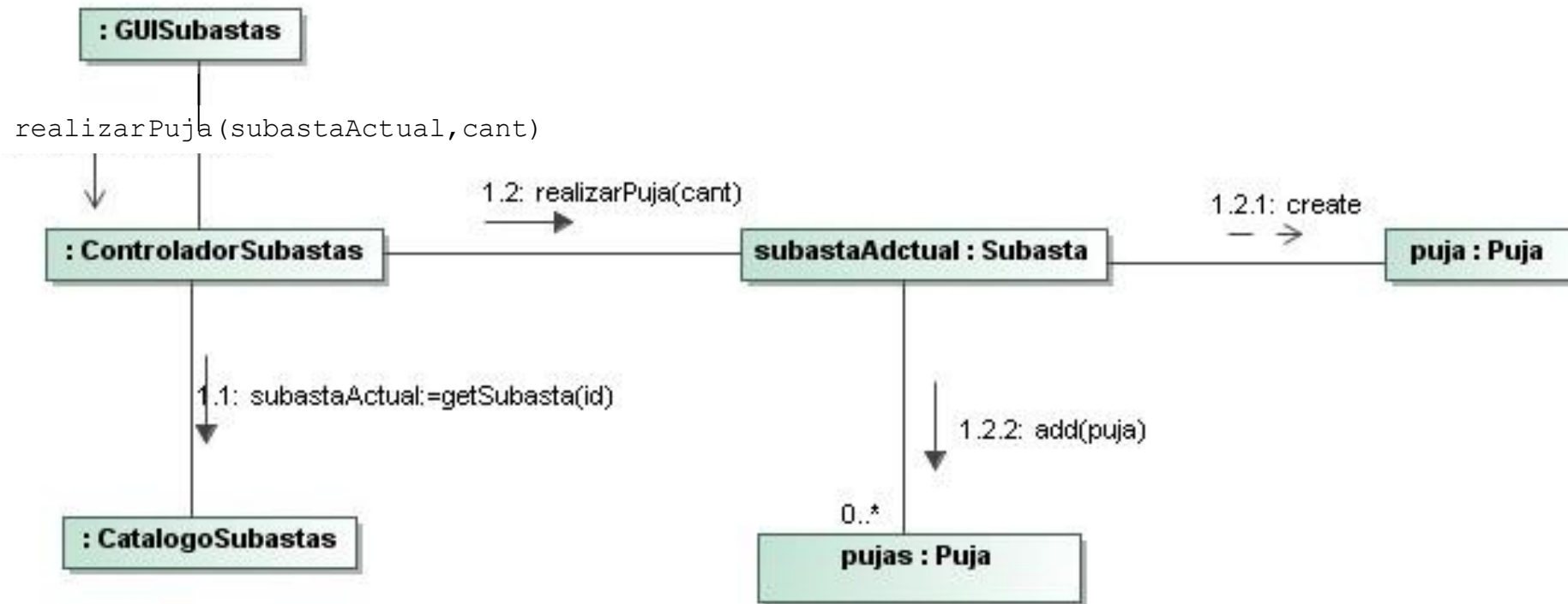
Ejemplo 2

- ¿Quién es responsable de crear una instancia de Puja ?



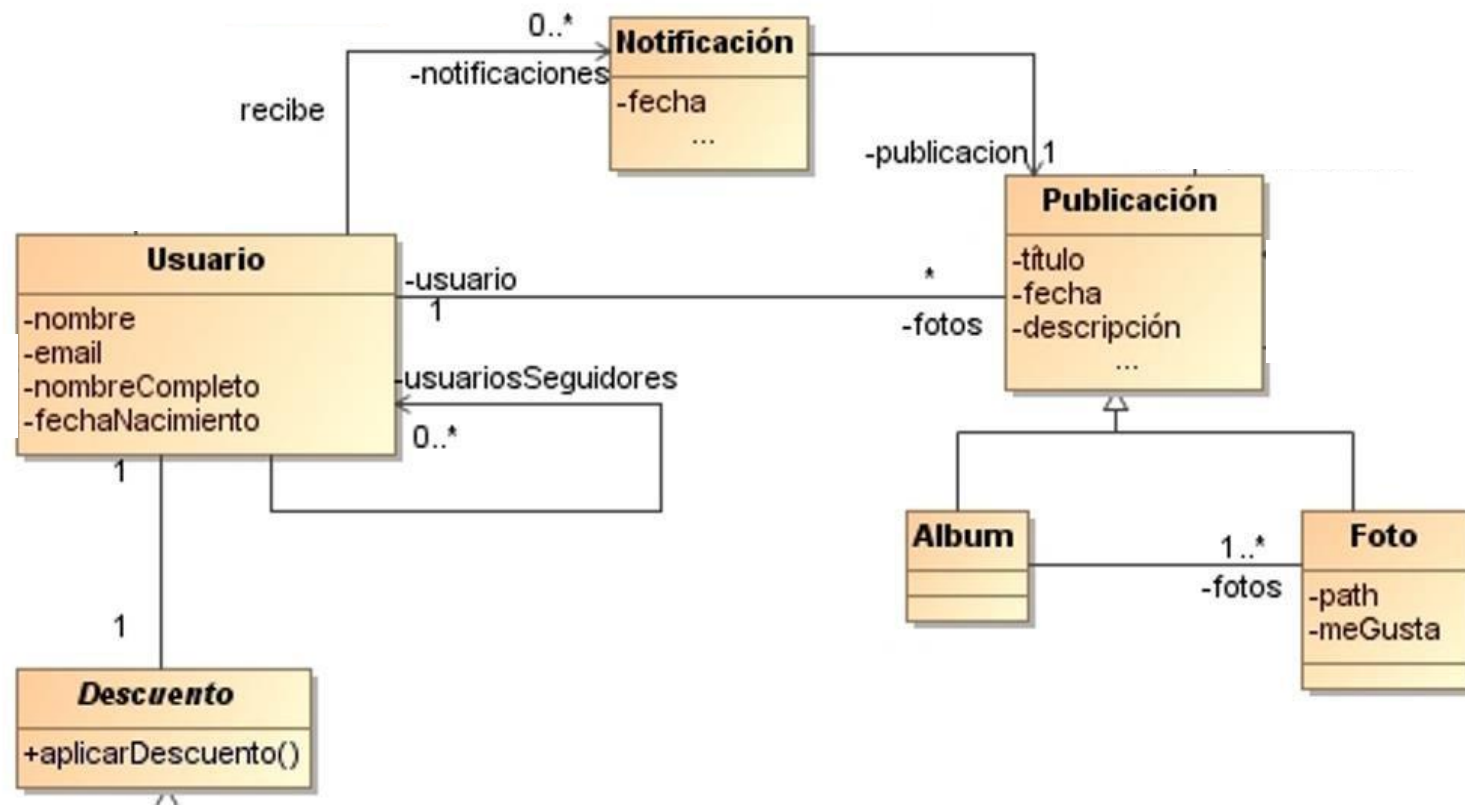
Ejemplo 2

- ¿Quién es responsable de crear una instancia de Puja?



Ejemplo 3

- ¿Quién es responsable de crear una instancia de Album o Foto?



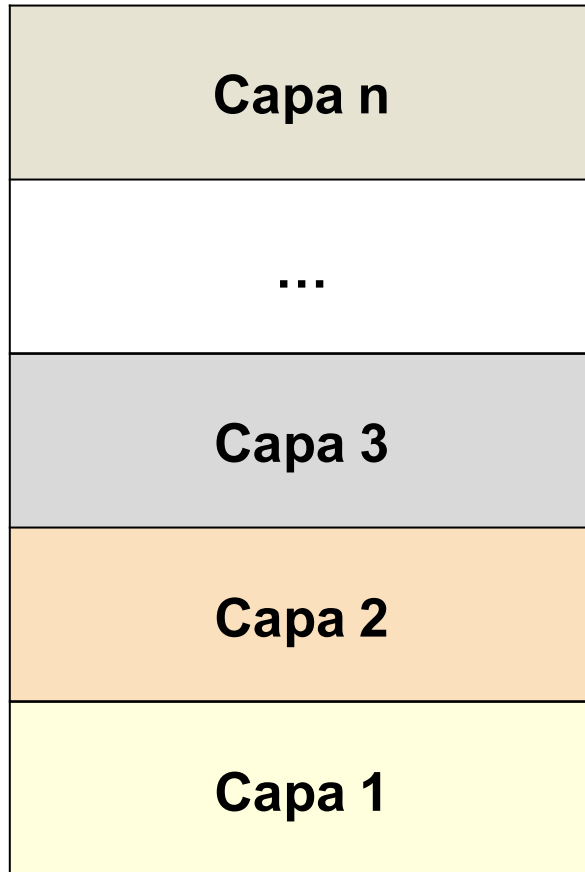
Controlador

¿Quién se encarga de atender los eventos del sistema (una GUI, normalmente)?

*Asignar responsabilidad de manejar eventos externos a uno o más objetos “**controlador**” que puede representar, por ejemplo, el sistema o una funcionalidad concreto (caso de uso o historia de uso).*

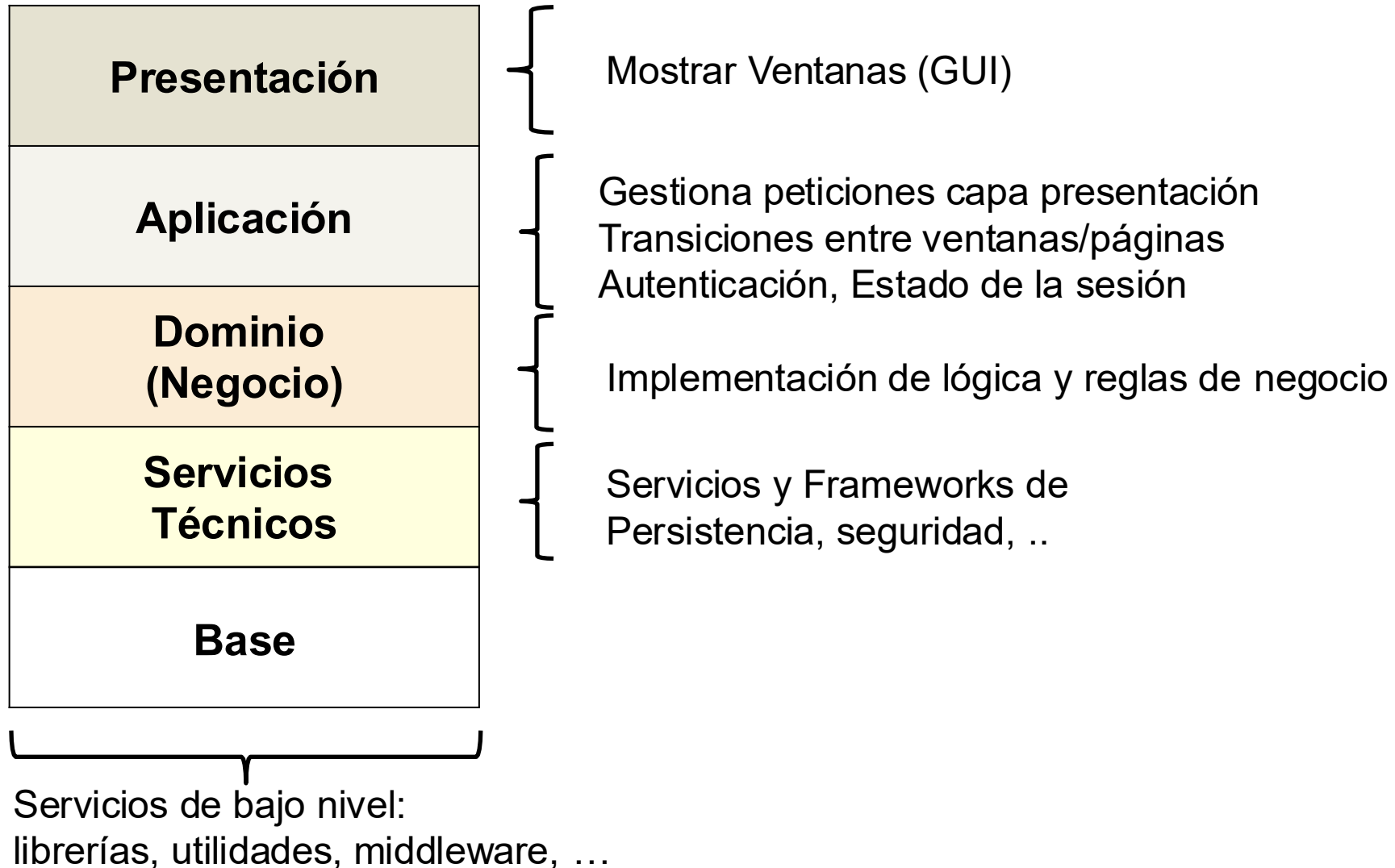
- ❑ **Fachada** entre clases de la capa presentación (GUI) y capa del dominio.
- ❑ Incluye, al menos, un método por cada evento atendida.
- ❑ **Beneficios** de un controlador:
 - **Separación modelo-vista**
 - Posibilidad de capturar información sobre el **estado de una sesión**

Arquitectura lógica organizada en “Capas” (Patrón arquitectural *Layers*)

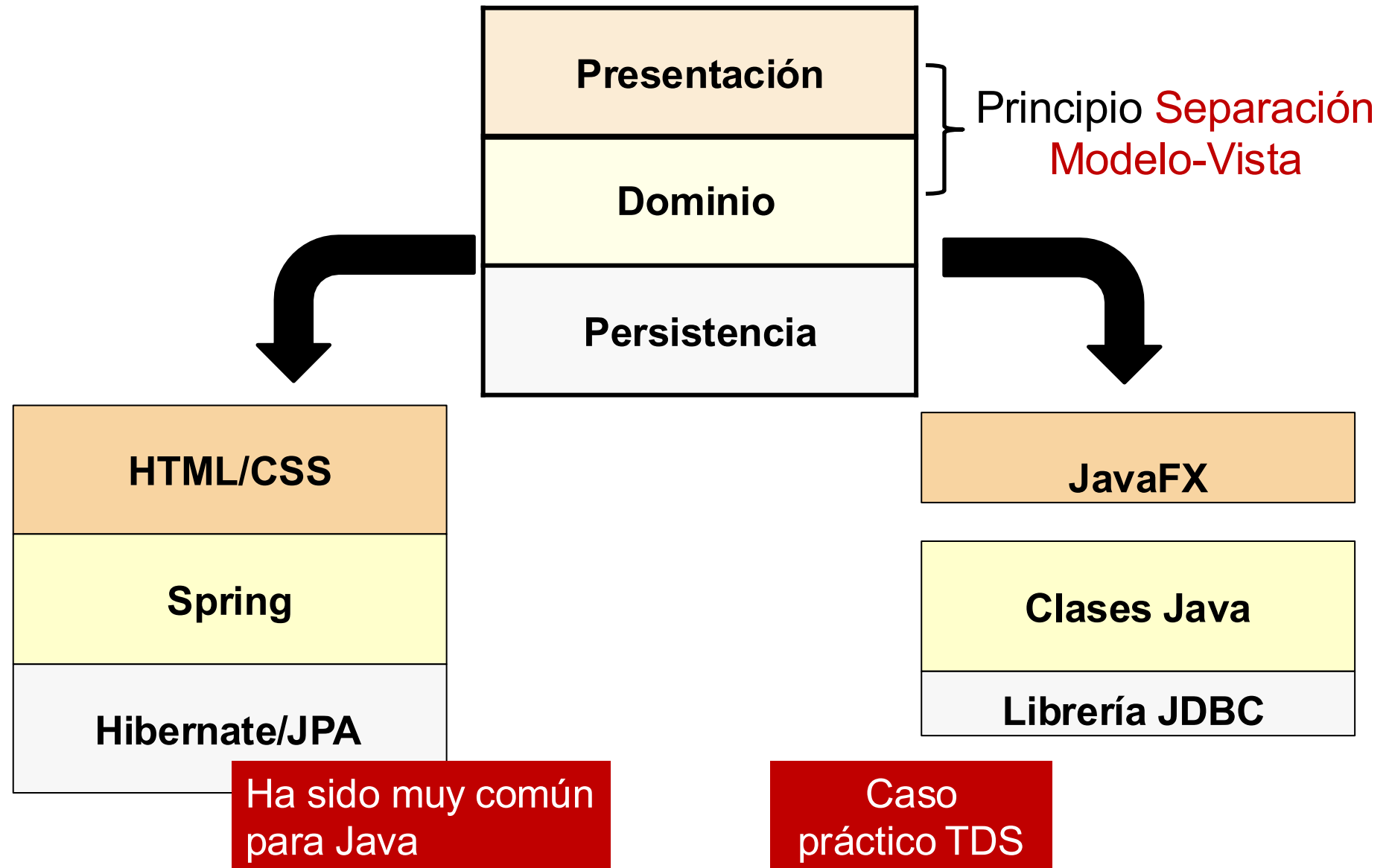


- Cada **capa** reúne clases relacionadas con un mismo **aspecto del sistema**, por ejemplo: interfaz de usuario, dominio o almacenamiento.
- **Utilidad**
 - Separación de aspectos favorece el cambio, la reutilización, y el desarrollo en paralelo.
 - Evita acoplamiento.

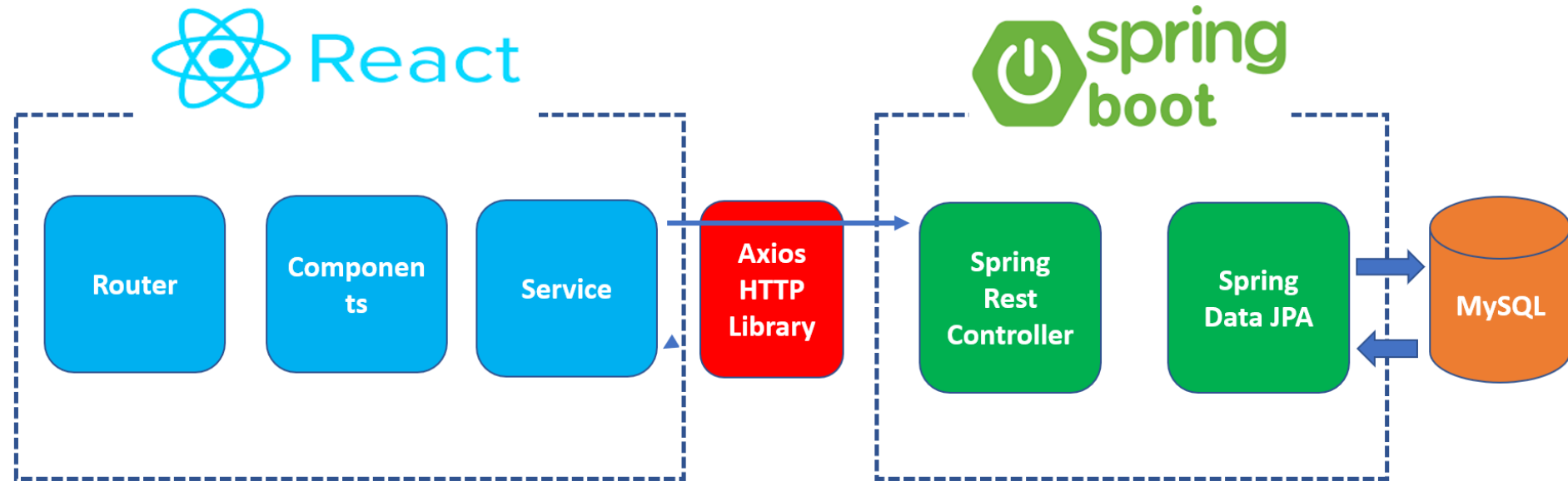
Arquitectura lógica organizada en 4-capas



Arquitectura de 3 capas: Muy extendida



ReactJS + Spring Boot CRUD Full Stack



By Ramesh Fadatore (Java Guides)

Front-end

Back-end



Arquitectura Limpia

La **Arquitectura Limpia** (Clean Architecture) es una propuesta de *Robert Cecil Martin (Uncle Bob)*. Se trata de un enfoque de diseño de software que se centra en la **separación de intereses** para crear **sistemas que son independientes de cualquier framework (ej: JavaFX), interfaz de usuario, bases de datos, y servicios externos.**

Su objetivo principal es hacer que el código sea más fácil de entender, mantener, probar y evolucionar

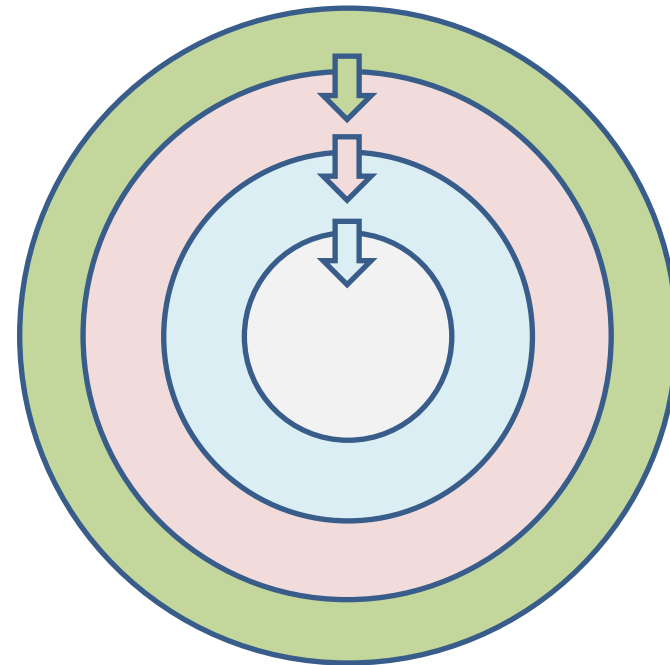
Referencia

Arquitectura Limpia

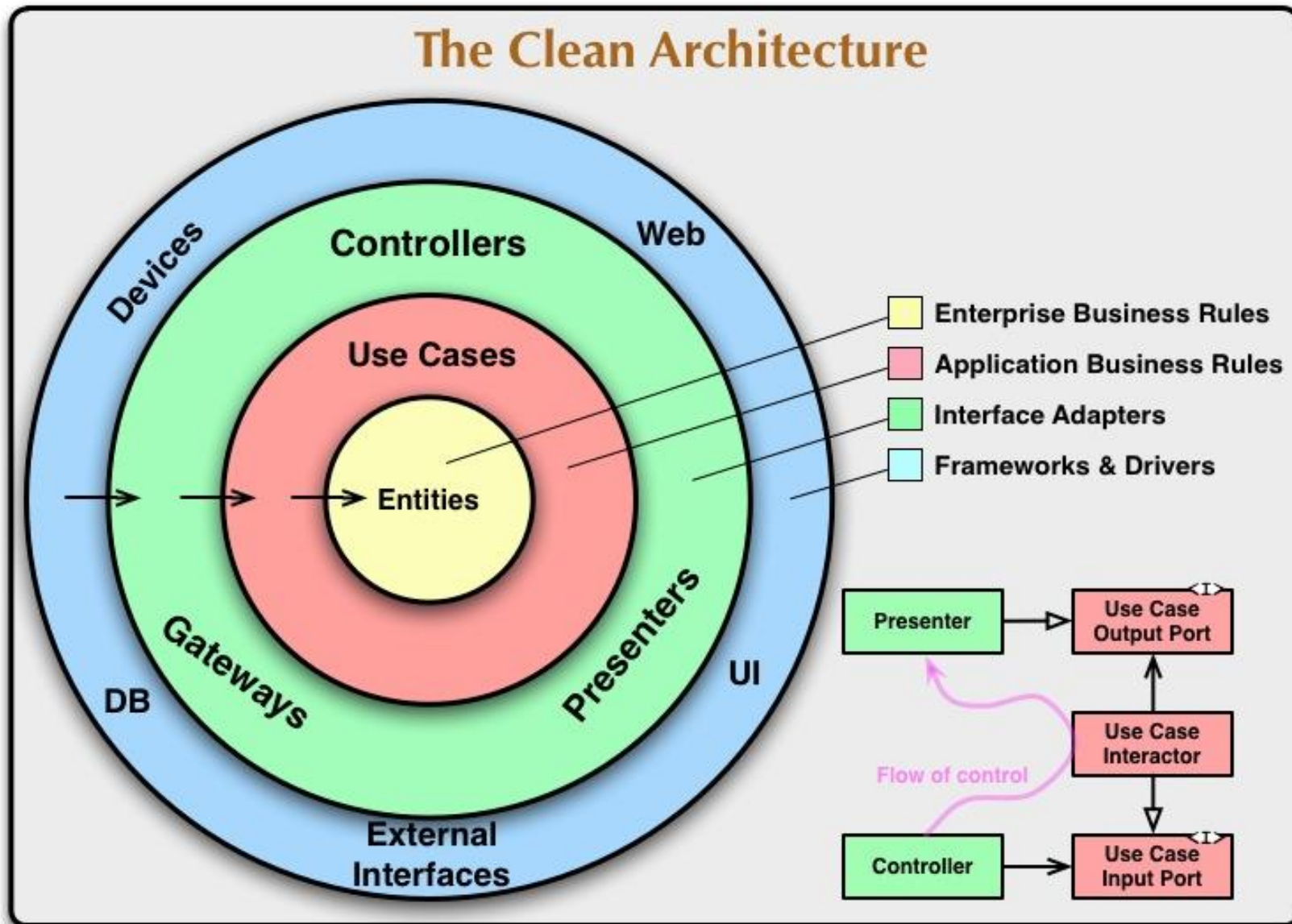
Se basa en el Principio de Inversión de Dependencias. Este principio establece que los módulos de alto nivel no deben depender de los módulos de bajo nivel.

La arquitectura se organiza en una serie de capas concéntricas, donde las capas internas no conocen nada de las capas externas.

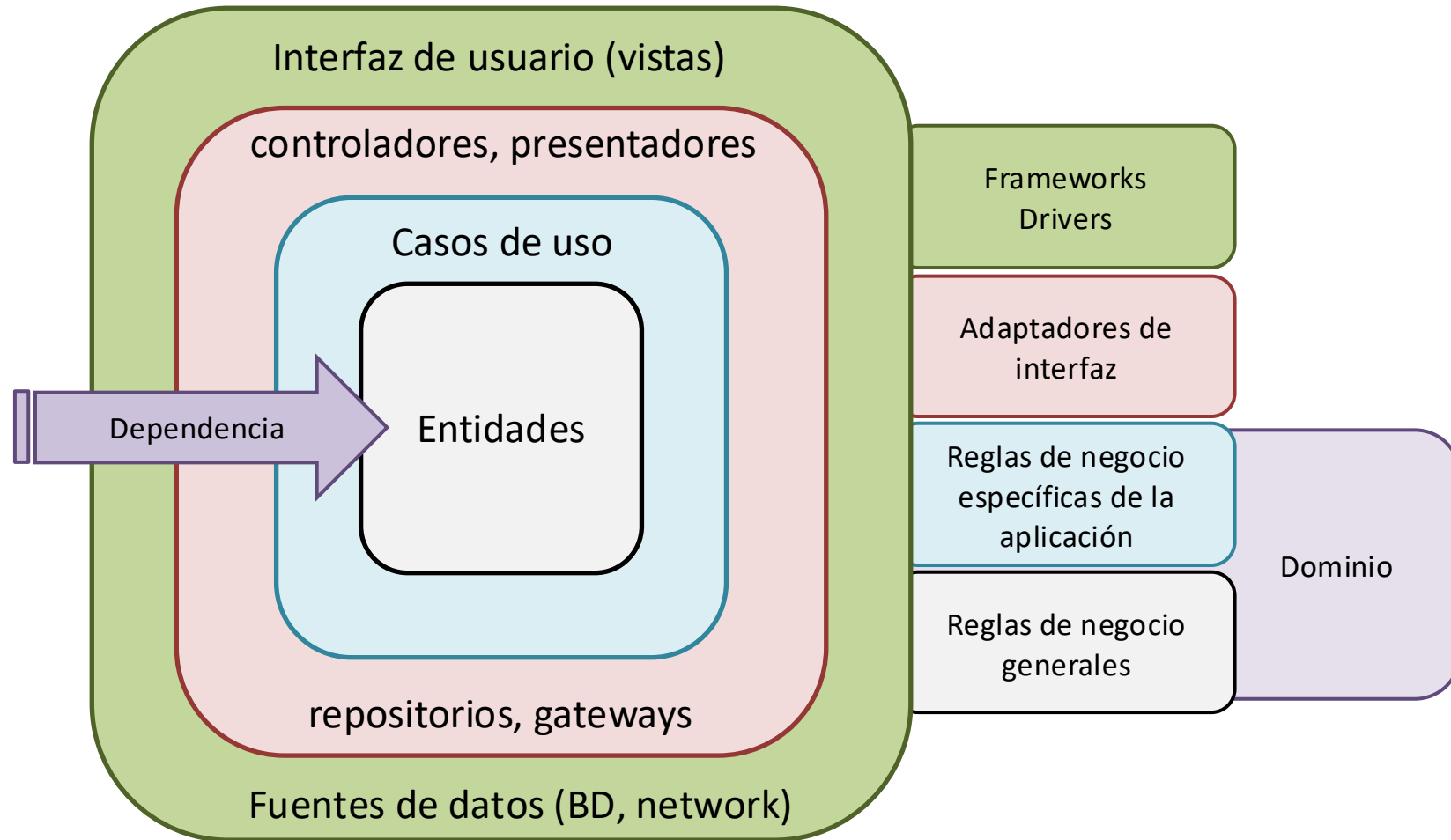
La dirección de las dependencias siempre va hacia el interior, es decir, **el código de las capas externas siempre depende del código de las capas internas, nunca al revés.**

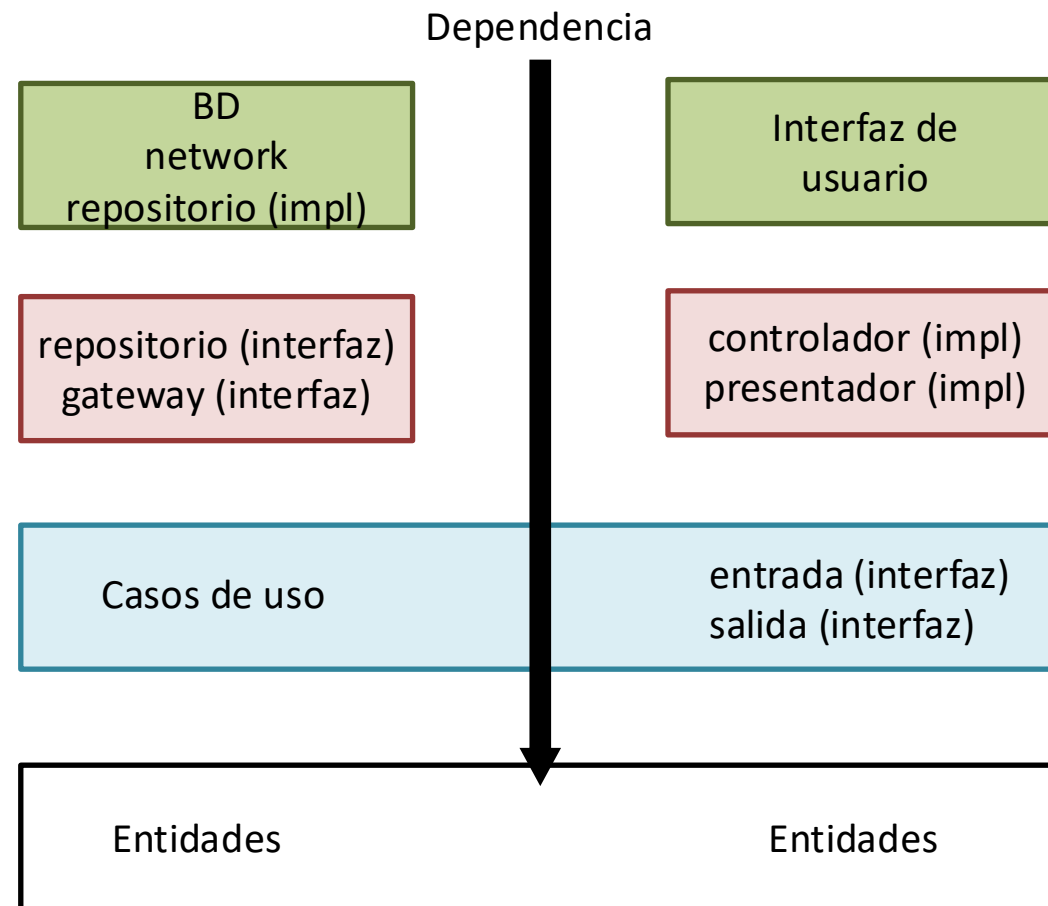
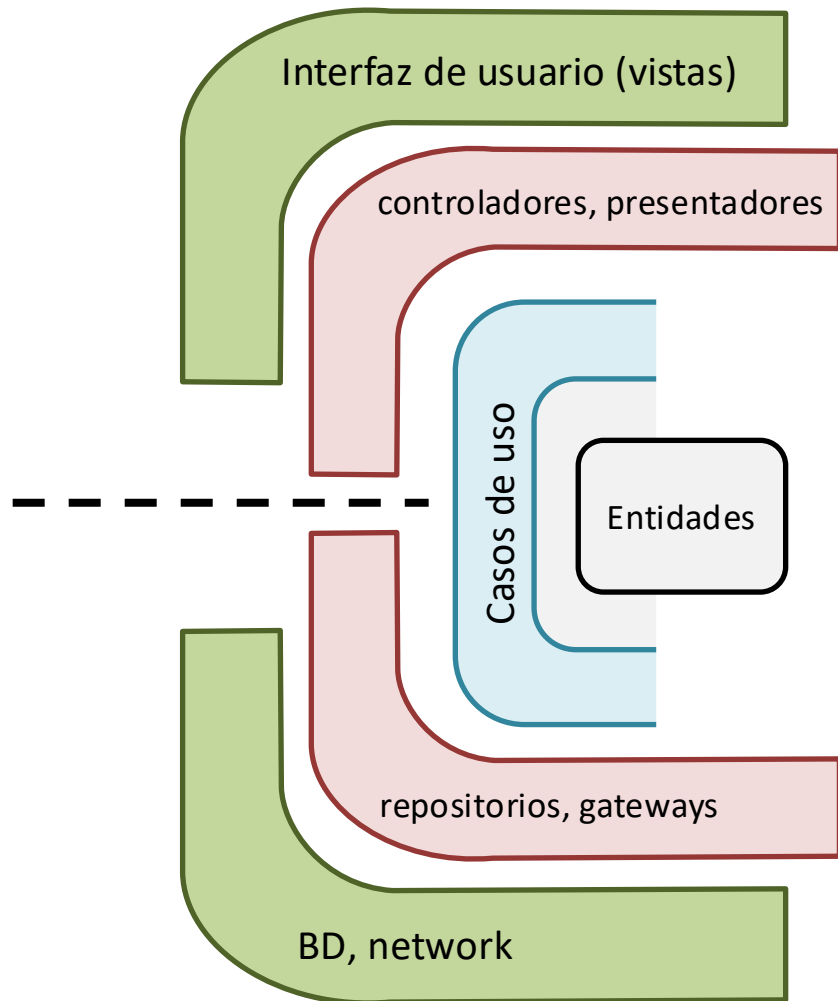


Arquitectura Limpia: Capas



Arquitectura Limpia: Capas

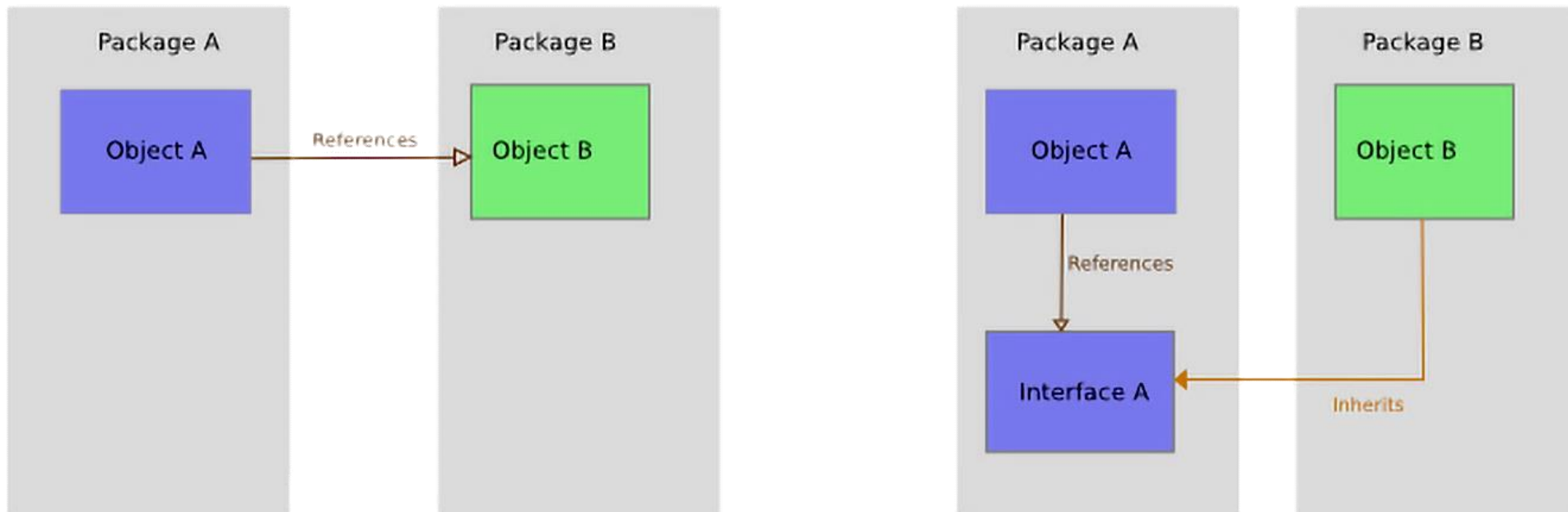


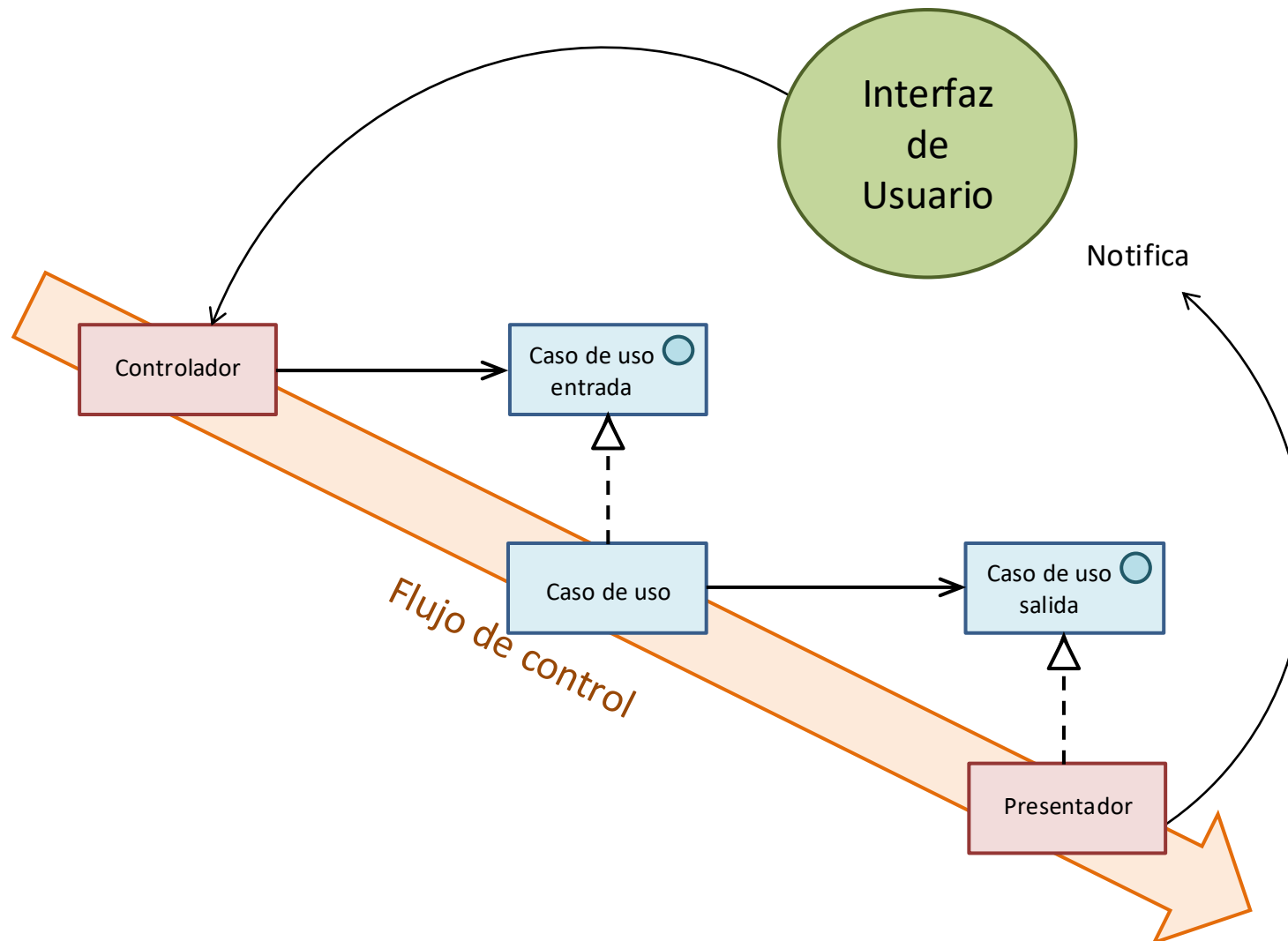


Arquitectura Limpia: inversión de dependencias

Para desacoplar los módulos, se aplica el **principio de inversión de dependencias**:

- Se introducen interfaces para establecer dependencias con abstracciones (Ej: sistema de persistencia)
- Las implementaciones de los interfaces concretan los detalles





Separación Modelo-Vista

Clases del Modelo

NO deben conocer/acceder a los objetos de la capa presentación (vista o interfaz).

Encapsulan la información y el comportamiento relacionado con la **lógica del negocio o de aplicación**.

Clases de la Vista

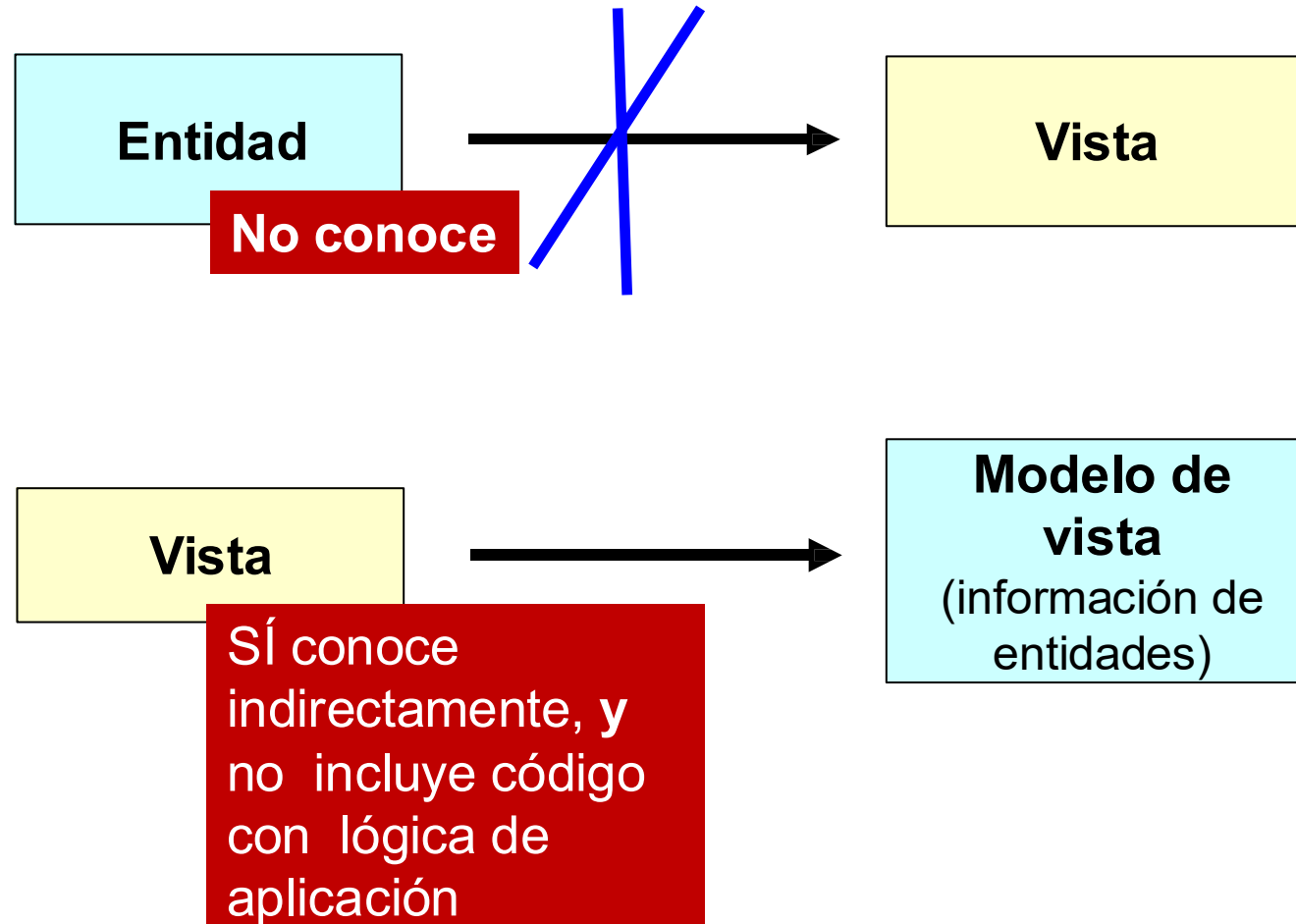
Son responsables de la entrada y salida, **capturando y manejando los eventos, pero**

NO encapsulan lógica del negocio/aplicación

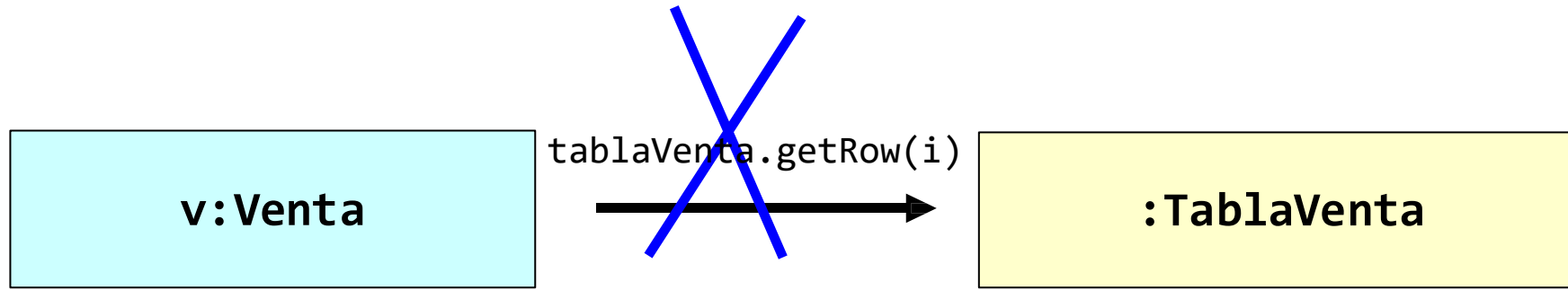
- - > **Delegan en un objeto controlador** que realiza operaciones ligadas a eventos.

¡OJO! Una clase de la vista puede invocar peticiones “get” sobre un **modelo de vista** de un objeto del dominio que le ha pasado el Controlador.

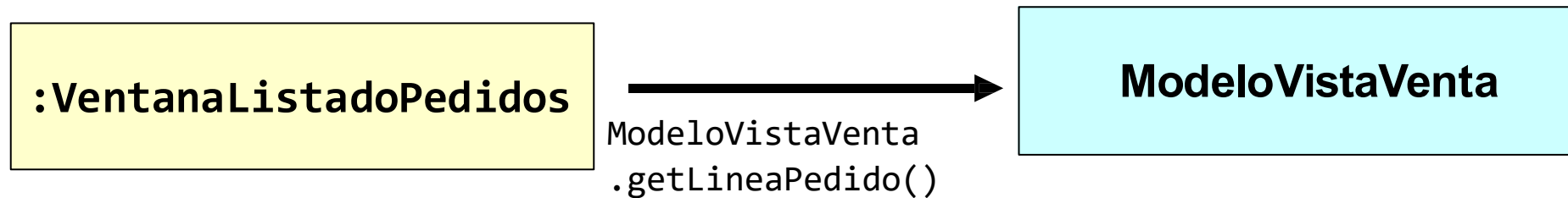
Separación Modelo-Vista



Separación Modelo-Vista



No sabe nada de ventanas



Una clase GUI no implementa lógica del negocio, por ejemplo getTotal()

Separación Modelo-Vista

La **capa presentación** no debería tener **ninguna responsabilidad de la lógica del negocio/aplicación**. Sólo debería ser responsable de las tareas propias de la interfaz de usuario: mostrar ventanas y captura eventos.

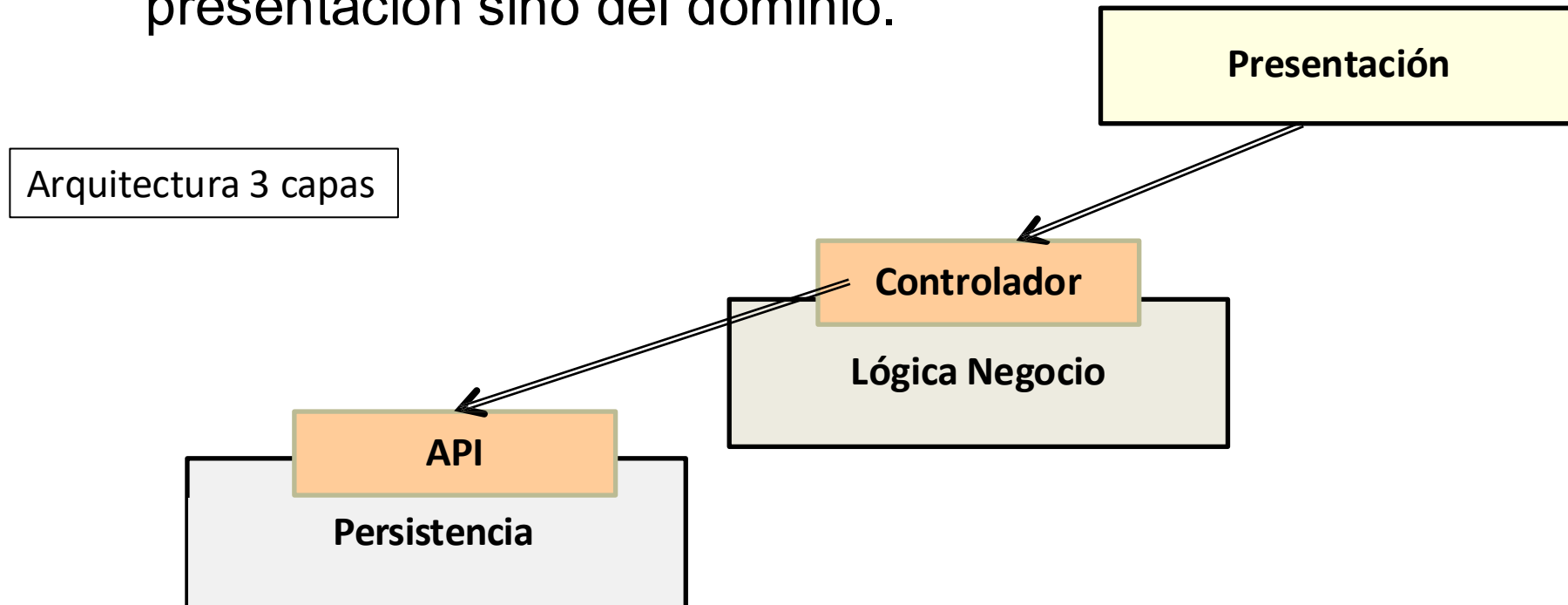
La **capa presentación** debería **remitir las responsabilidades** propias de las capas del dominio a esas capa, **a través de un controlador**.

Separación Modelo-Vista: **Ventajas**

- Favorece clases **cohesivas**
- Permite **desarrollar al mismo tiempo** las clases de la vista y del dominio
- Minimiza el impacto de los **cambios** en la interfaz sobre las clases del modelo.
- Facilita conectar otras vistas a una capa del dominio existente (**reutilización**).
- Permite varias **vistas simultáneas** sobre un mismo modelo.
- Permite que la capa del modelo se **ejecute** como un **proceso independiente** a la capa de presentación.

Controlador (retomamos)

- Las clases de la interfaz delegan en un **controlador** la realización de las tareas ligadas a cada evento del sistema que procede, normalmente, de la GUI.
- El controlador es una “**fachada**” entre las clases de la vista y del dominio: **Separar interfaz de usuario de la lógica del dominio o de la aplicación**. No pertenece a la capa presentación sino del dominio.



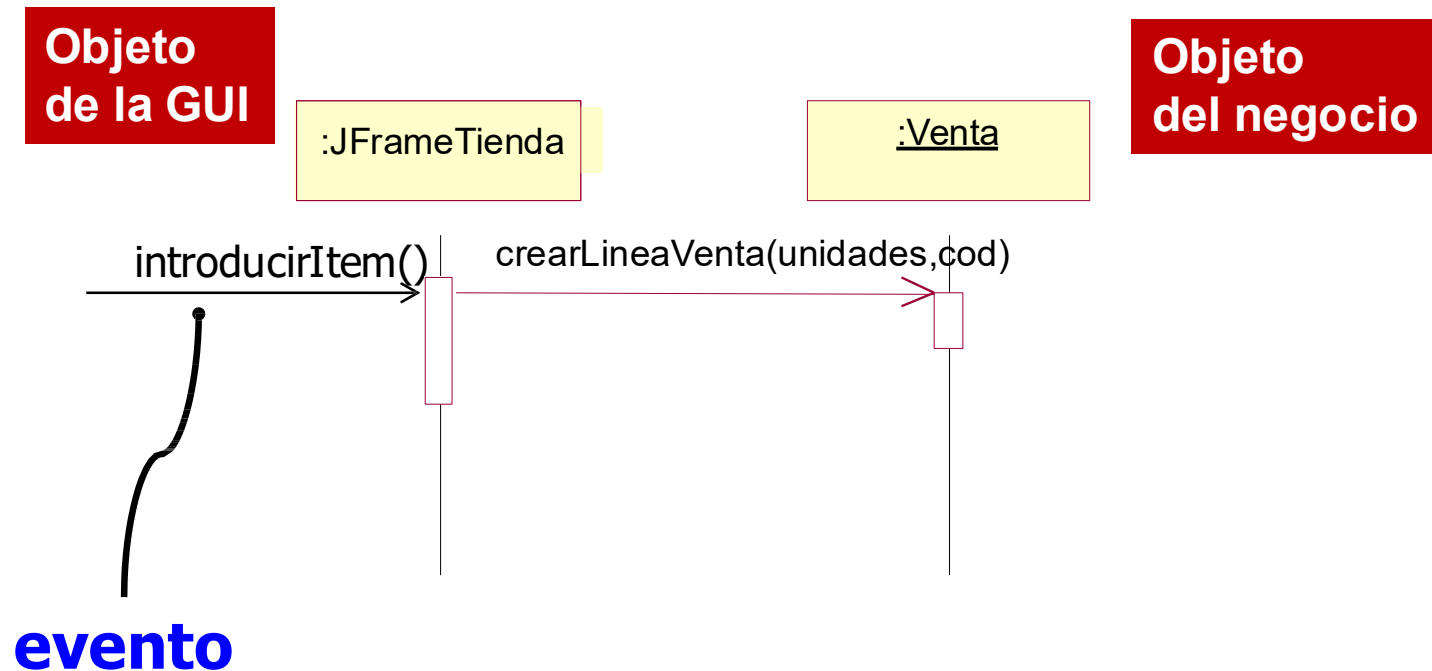
```
public class JFrameTienda {..  
    public void initialize(){  
        //manejador evento clic en boton para registrar venta  
        btnRegistrar.addActionListener(e -> {  
            Producto producto = productoCombo.getSelectedItem();  
            Integer unidades =Integer.parseInt(unidades.getText());  
            ..  
            // delega en controlador  
            ControladorTienda.getUnicaInstancia().  
                crearLineaVenta(unidades,producto);  
        }  
    }  
}
```

Ventana

```
public class ControladorTienda{..  
    public void anadirLineaVenta(int unidades, Producto producto) {  
        ventaActual.addLineaVenta(unidades, producto);  
    }  
    public void registrarVenta(){..}
```

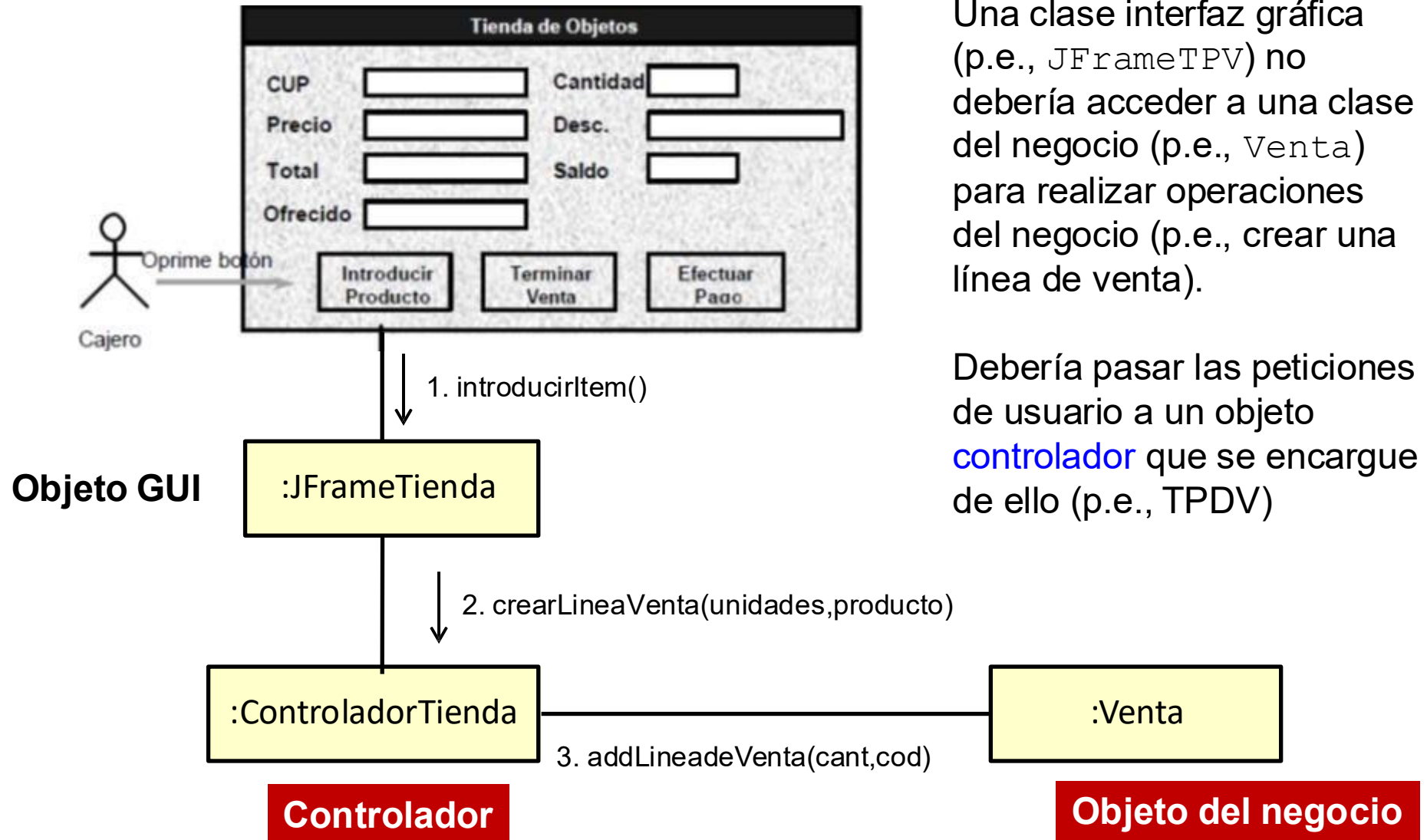
Controlador

NO separación modelo-vista

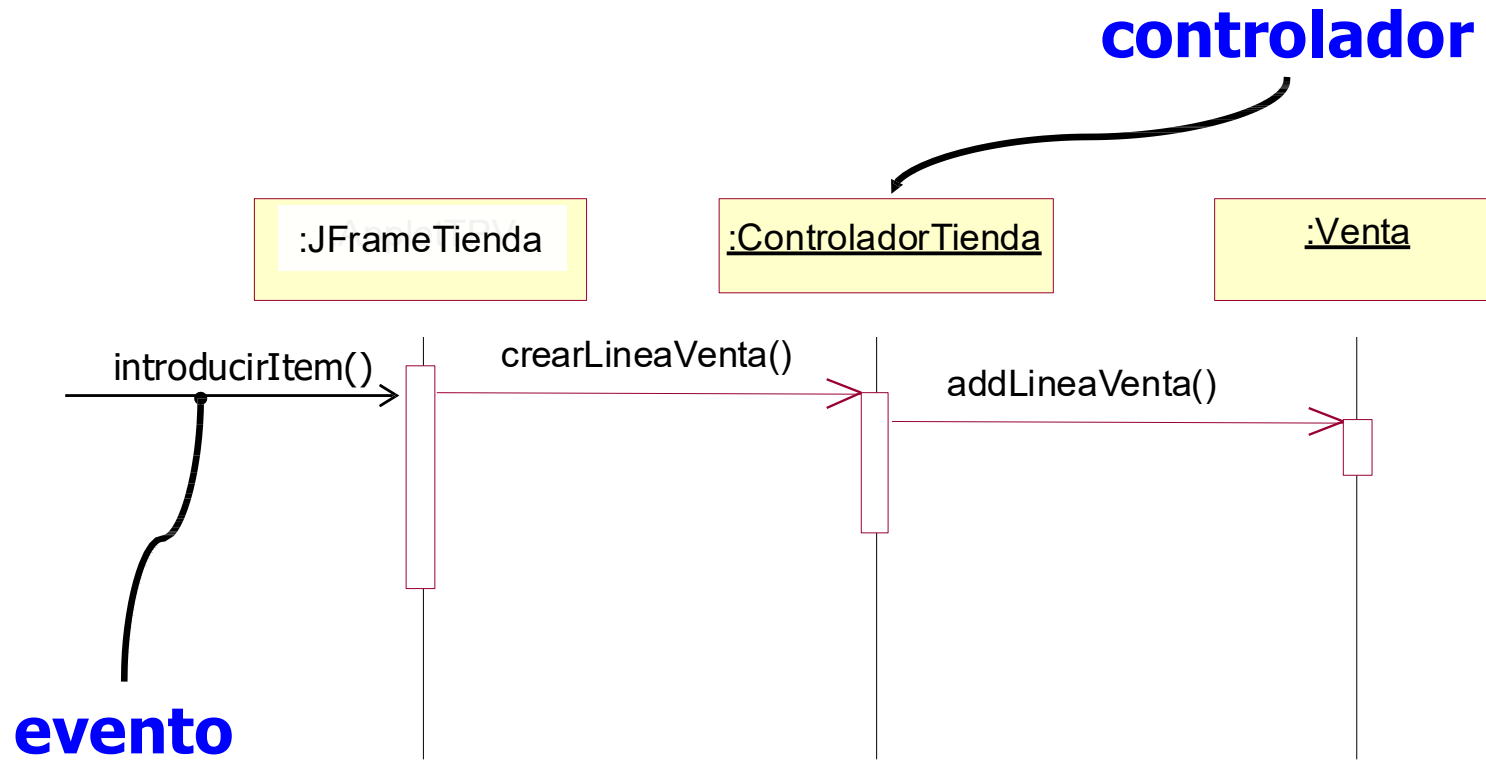


Acoplamiento inadecuado de la capa presentación
con la capa del dominio

Ejemplo de Separación modelo-vista



Ejemplo de Separación modelo-vista



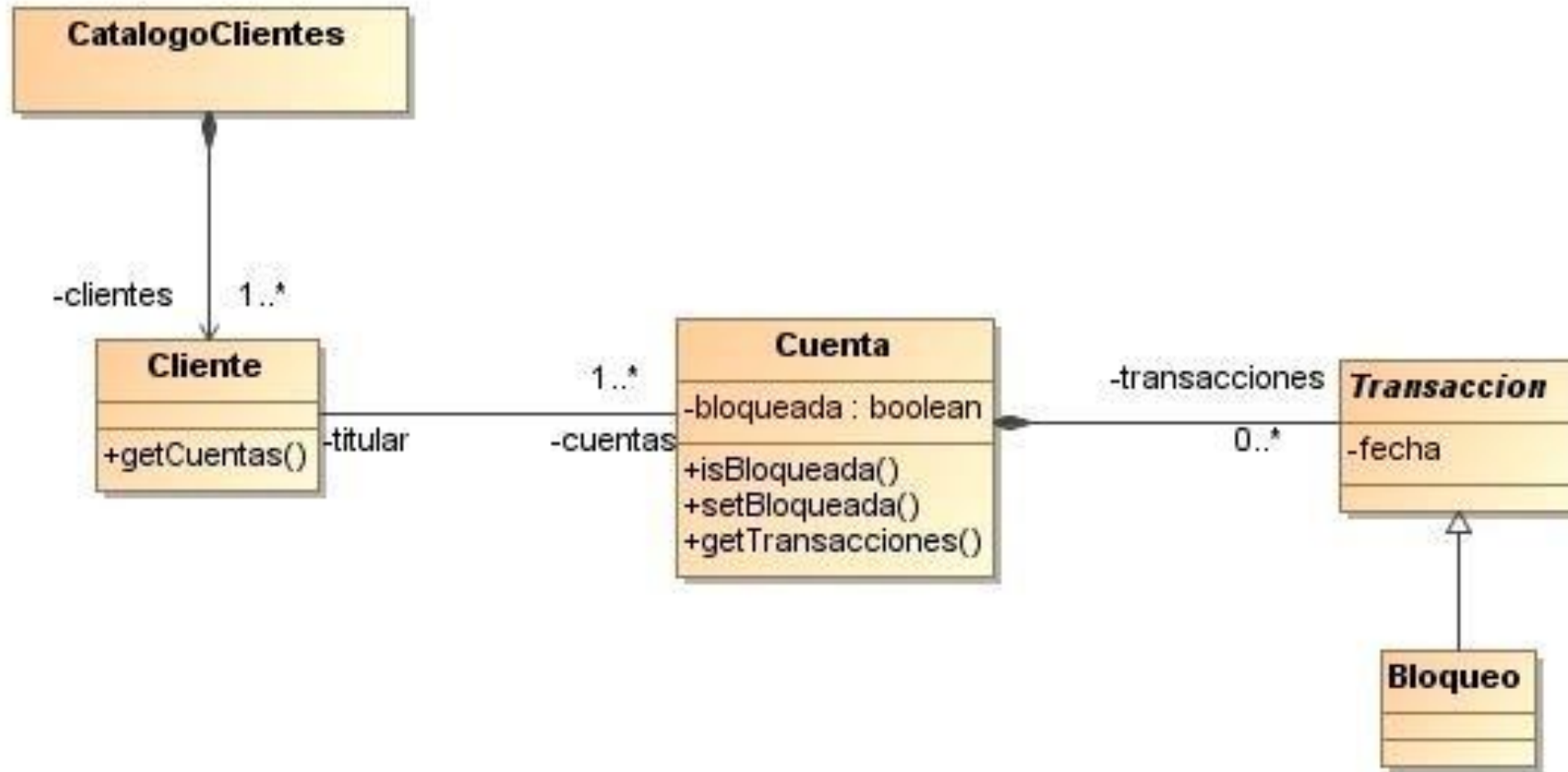
Acoplamiento adecuado de la capa presentación con la capa del dominio

Ejercicio. Modificar código para que no se viole ningún patrón GRASP

El siguiente código es parte de una aplicación bancaria que dispone de la funcionalidad de **bloquear todas las cuentas de un cliente** a petición de Hacienda.

- La clase `ViewGestionCuentas` implementa una ventana a través de la cual se pueden realizar diversas operaciones sobre las cuentas de un cliente.
- La clase `Cliente` tiene un atributo `cuentas` de tipo `ArrayList<Cuenta>` que registra todas las cuentas de ese cliente y el método `getCuentas()` retorna esa colección.
- La clase `Cuenta` tiene un atributo “bloqueada” de tipo booleano que determina si una cuenta está bloqueada y el método `setBloqueada()` que establece su valor.
- Al bloquear la cuenta se crea una instancia de la clase `Bloqueo` que es una subclase de la clase abstracta `Transaccion`. Esta clase tiene un atributo para registrar la fecha actual y sus subclases añaden atributos según el tipo de transacción.
- La clase `Cuenta` también tiene un atributo `transacciones` de tipo `ArrayList<Transaccion>` que registra todas las transacciones realizadas sobre una cuenta y el método `getTransacciones()` que retorna dicha colección.
- La clase `CatálogoClientes` es una clase que registra la colección de clientes.

Ejercicio. Modificar código para que no se viole ningún patrón GRASP



Ejercicio. Modificar código para que no se viole ningún patrón GRASP

```
public class ViewGestionCuentas extends JFrame {
    private Cliente cliente;
    private JTextField campoDNI;
    ...
    public ViewGestionCuentas() {
        ...
        JPanel panel = new JPanel();
        JLabel rotuloDNI = new JLabel("DNI Cliente");
        campoDNI = new JTextField();
        campoDNI.setText("0");
        panel.add(rotuloDNI);
        panel.add(campoDNI);
        ...
        JButton botonBloquear = new JButton("Bloquear");
        botonBloquear.addActionListener(new ActionListener()
        {
            public void actionPerformed(ActionEvent ev) {
                Cliente cliente = CatalogoClientes.getInstancia().
                    getCliente(campoDNI.getText());

                if (cliente != null){
                    ArrayList<Cuenta> cuentas = cliente.getCuentas();
                    for (Cuenta cuenta: cuentas) {
                        cuenta.setBloqueada(true);
                        ArrayList<Transaccion> transacciones =
                            cuenta.getTransacciones();
                        transacciones.add(new Bloqueo(new Date()));
                    }
                } else
                    JOptionPane.showMessageDialog(ViewGestionCuentas.this,
                        "No existe cliente con ese dni");
            }
        });
        ...
    }
}
```

Ejercicio. Modificar código para que no se viole ningún patrón GRASP

```
public class ViewGestionCuentas {..  
    botonBloquear.addActionListener(ev ->  
        Cliente cliente = CatalogoClientes.getInstance().  
            getCliente(campoDNI.getText());  
  
        if (cliente != null)  
            AppCuentas.getInstance().bloquearCuentas(cliente);  
        else JOptionPane.showMessageDialog(ViewGestionCuentas.this,  
            "No existe cliente con ese dni");  
    }  
}  
  
public class AppCuentas {..  
    //controlador de la aplicación  
    public int bloquearCuentas(Cliente cliente) {  
        cliente.bloquearCuentas();  
    }  
}
```

Ejercicio. Modificar código para que no se viole ningún patrón GRASP

```
public class Cliente {..  
    private ArrayList<Cuenta> cuentas;  
    ..  
    public int bloquearCuentas() {  
        for (Cuenta cuenta: cuentas)  
            cuenta.bloquear();  
    }  
}  
  
public class Cuenta {..  
    private boolean bloqueada;  
    private ArrayList<Transaccion> transacciones;  
    ..  
    public void bloquear(){  
        this.bloqueada = true;  
        transacciones.add(new Bloqueo(LocalDate.now()));  
    }  
}
```

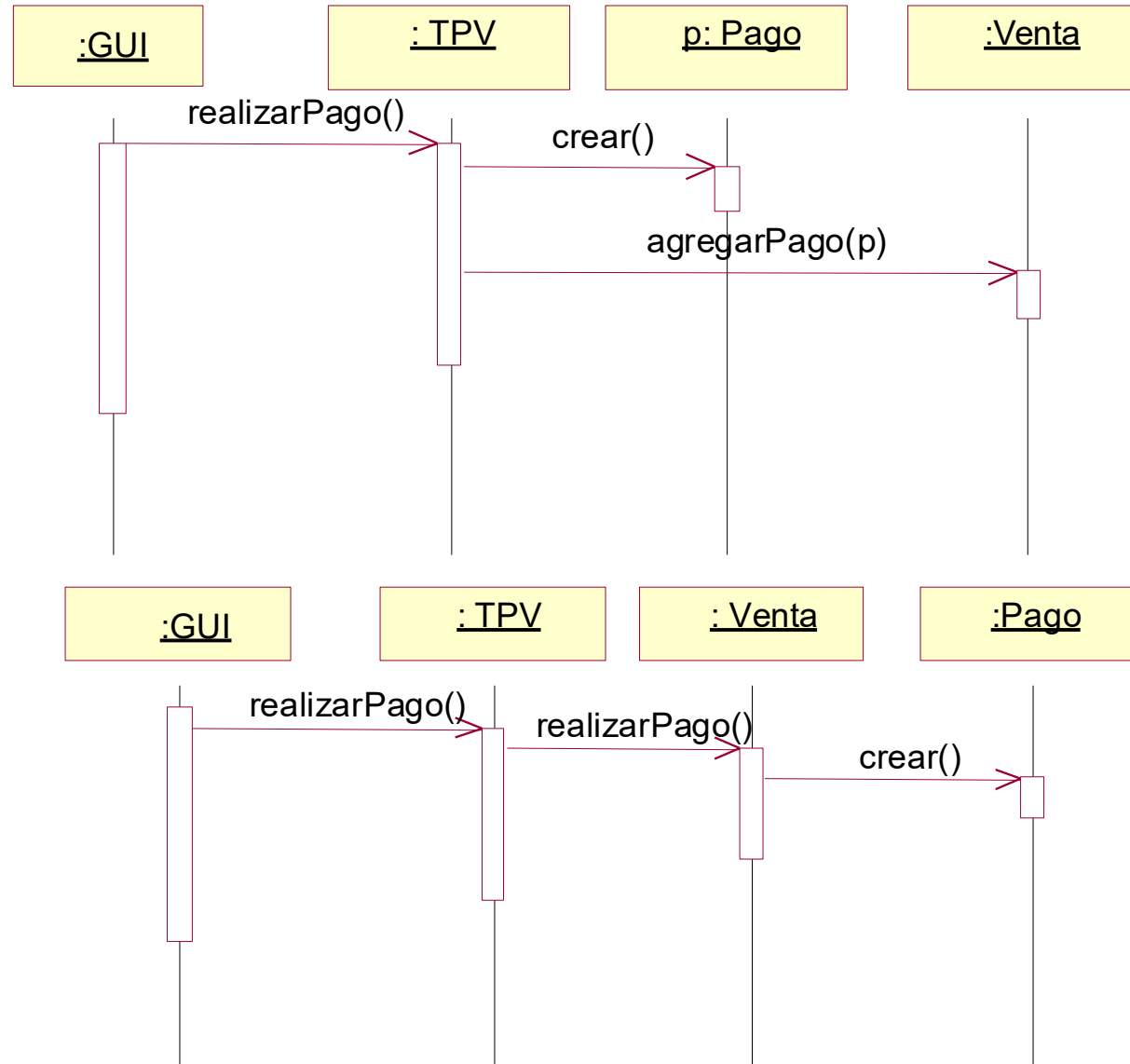
Bajo Acoplamiento

¿Cómo reducir las dependencias entre clases?

“Asignar responsabilidad de modo que se consiga un bajo acoplamiento”

- No considerarlo de forma independiente, sino junto a los patrones Experto y Alta Cohesión.
- **Beneficios:** Facilita i) reutilización, ii) comprensión de las clases y iii) mantenimiento
- Es un patrón **evaluativo**
- ¿Cuándo existe acoplamiento entre una clase A y otra B?
 - A posee un **atributo** o campo de tipo *B*
 - A tiene un **método** con un **parámetro**, una **variable local** o **valor de retorno** de tipo *B*.
 - A es **subclase directa o indirecta** de *B*
 - A **implementa** la interfaz *B* (Java)

Ejemplo simple de evitar acoplamiento



Alta Cohesión

¿Cuánto están relacionadas las responsabilidades de una clase?

- “ Asignar una responsabilidad de modo que la cohesión siga siendo alta”
- “ Una clase debería representar **una única abstracción bien definida** que tiene un conjunto de métodos muy relacionados funcionalmente.

- **Baja Cohesión:**

- Clases con responsabilidades que **deberían haber delegado** en otras.
- Son “**clases dios**” con gran cantidad de métodos y se pueden establecer grupos de ellos que no están fuertemente relacionados funcionalmente .
- Difíciles de comprender, reutilizar, mantener -> **Beneficios** de aplicar el patrón.

Ejemplos de cohesión

- **Muy baja** cohesión:
 - Clase que mezcla funcionalidad de **acceso a bases de datos** con **acceso remoto** a través de RMI.
- **Baja** cohesión:
 - **Una única clase AccesoBD** con toda la funcionalidad relacionada con el acceso a una base de datos tendría muchos métodos: debería representar conexiones, sentencias sql, transacciones, etc.
- **Alta** cohesión:
 - **Clase AccesoBD + Familia de clases relacionadas** (Ejemplo JDBC, Hibernate) - - > librería o framework de clases

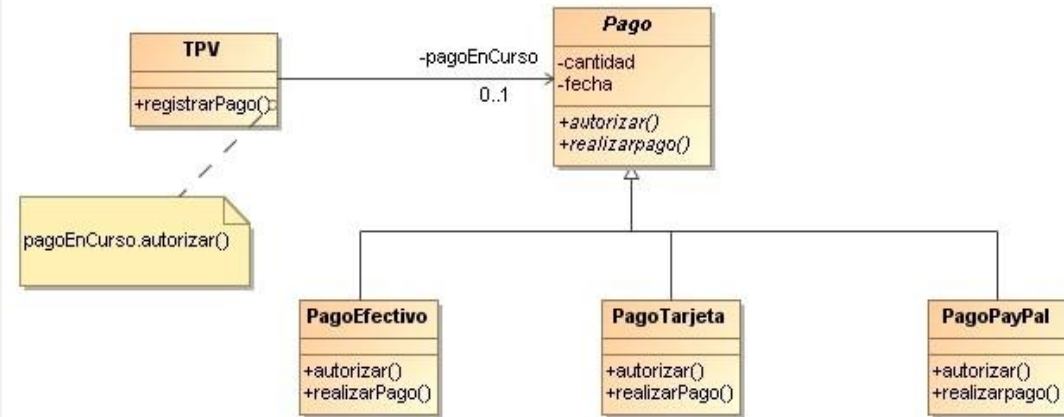
Polimorfismo (I)

¿Cómo manejar las alternativas basadas en el tipo?

Existe comportamiento que depende de una variación de tipos.

No realizar un análisis de casos basado en el tipo de los objetos, sino definir una jerarquía de herencia -> Asignar la responsabilidad de cada comportamiento al correspondiente tipo (**Curso de POO básico**).

```
switch (tipoPago) {  
    case "Tarjeta":  
        autorizarPagoTarjeta();  
        break;  
    case "Paypal":  
        autorizarPagoPaypal();  
        break;  
    case "Transferencia":  
        autorizarPagoTransferencia();  
        break;  
    default:  
        // ...  
}
```



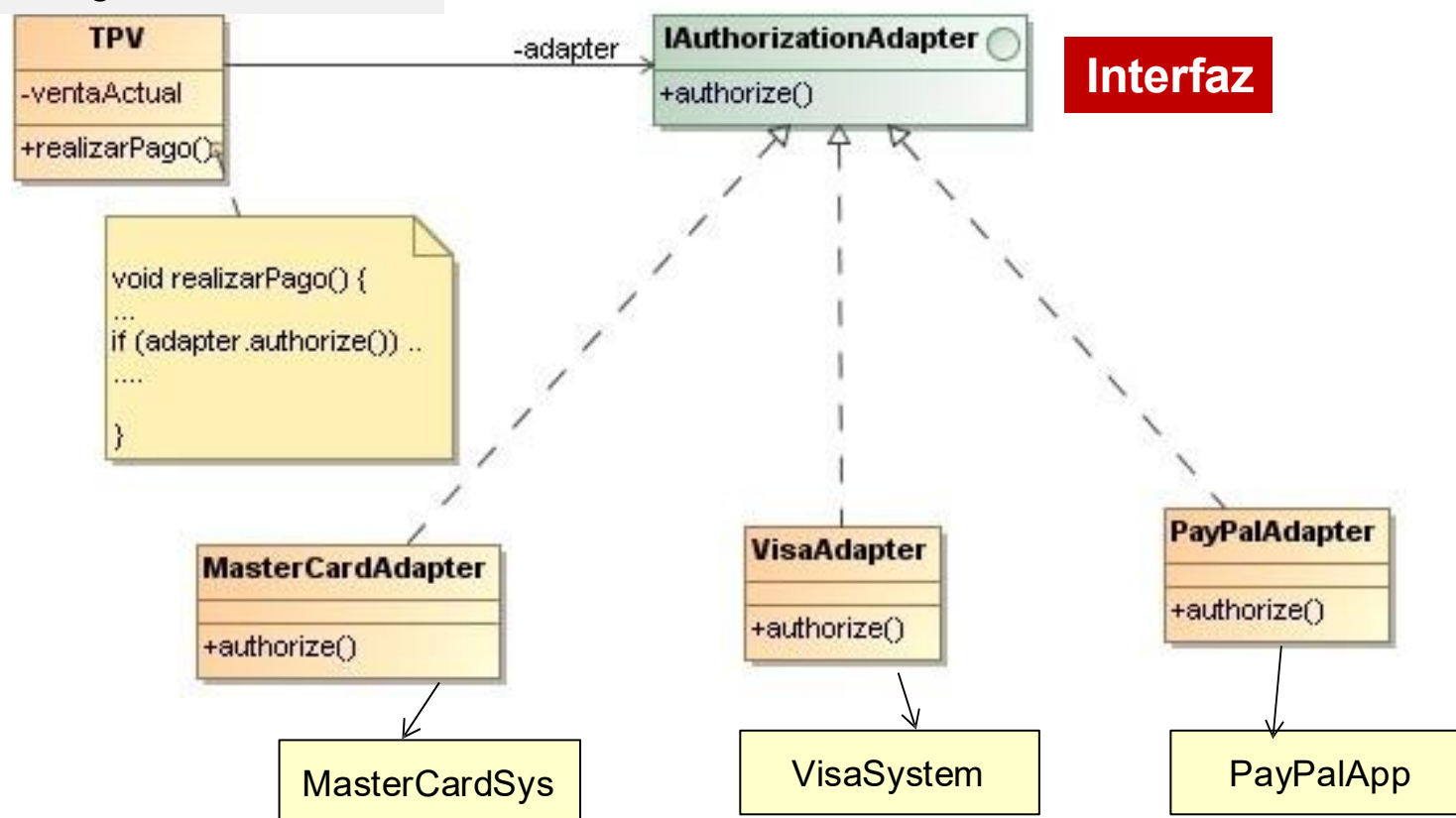
`pagoEnCurso.autorizar()`

Polimorfismo (II)

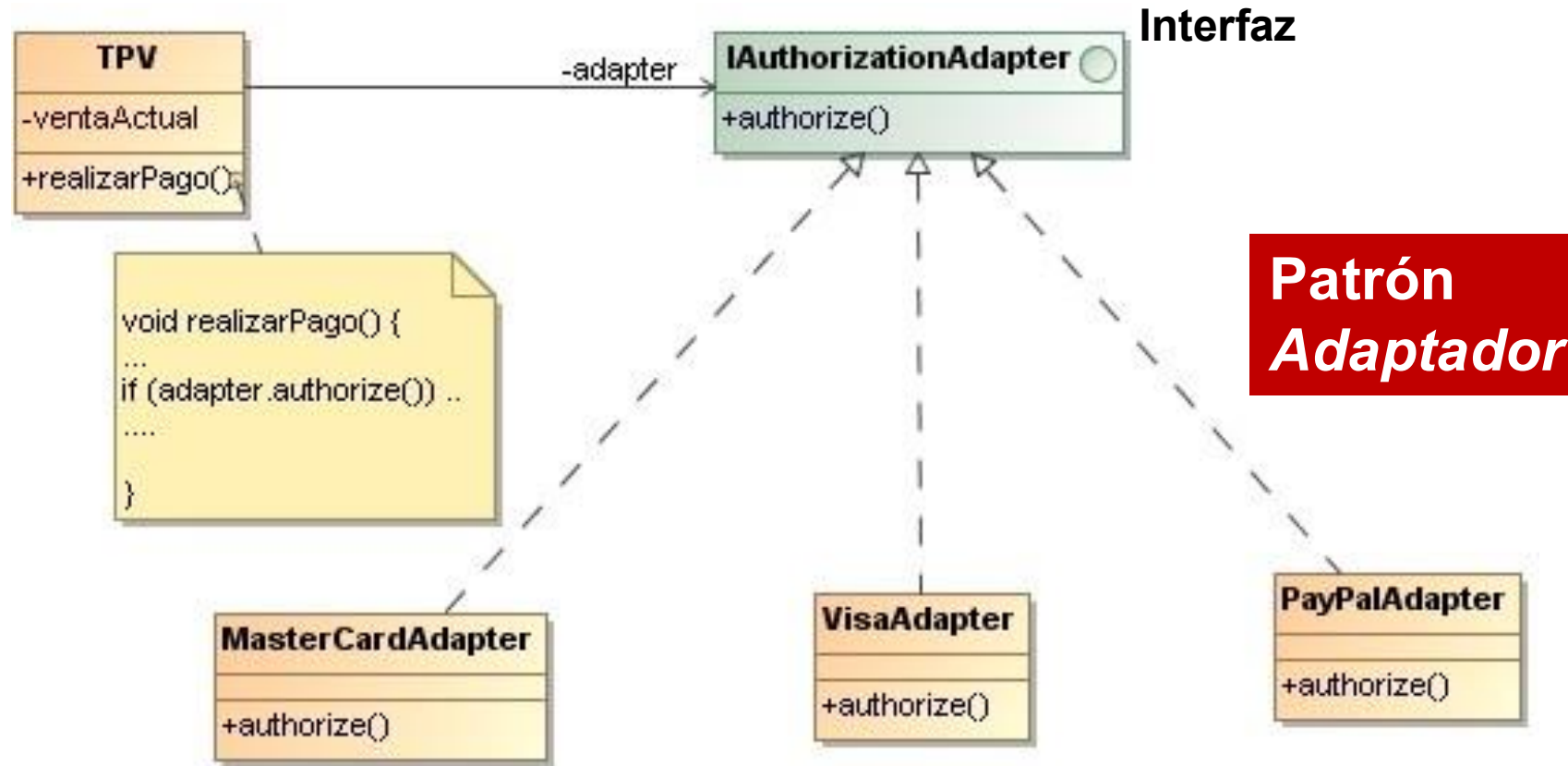
¿Cómo conseguir que una clase pueda interactuar con clases no previstas que ofrecen una funcionalidad que requiere?

Definir una **interfaz** que proporcione los métodos que permitan la interacción sin conocer clases concretas.

App no debe estar ligada a un *Sistema de Pago* concreto



Polimorfismo (III)

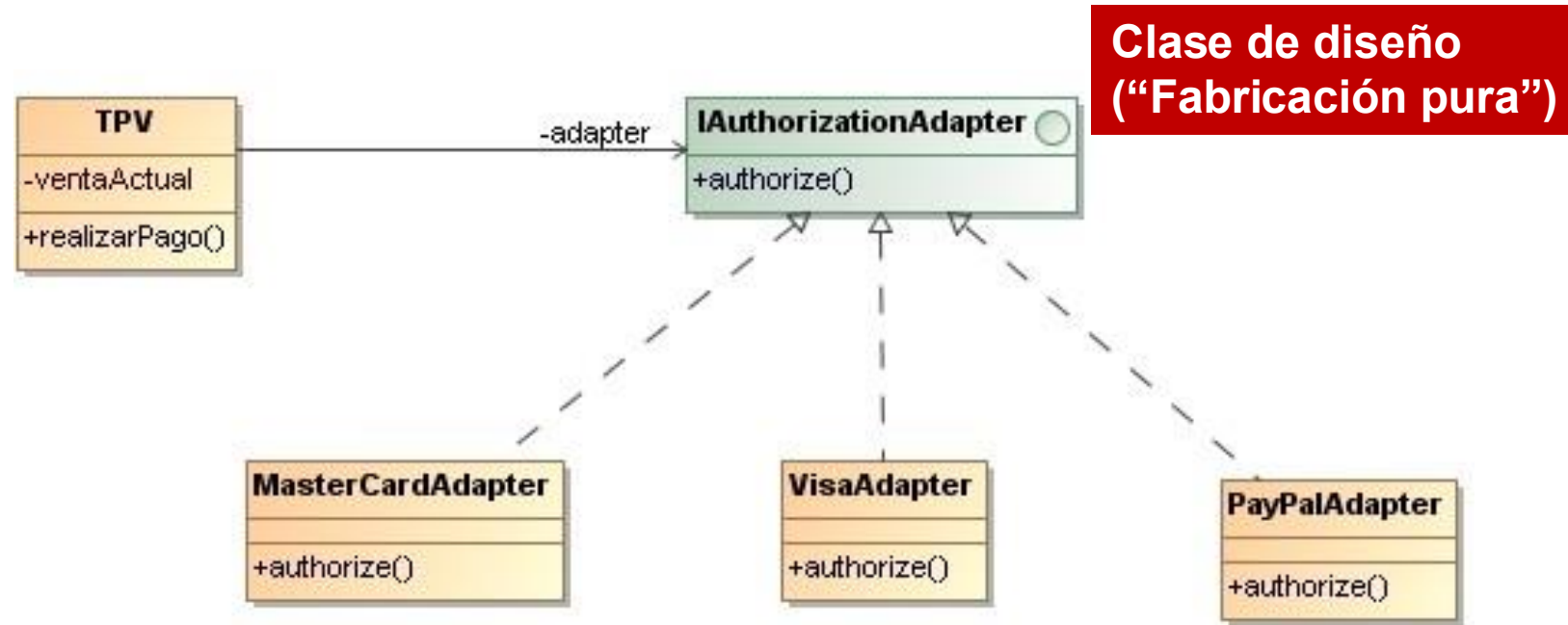


- ✓ Las **interfaces** son claves para la “pluggability”
 - Se añaden fácilmente nuevas alternativas
 - Los clientes no conocen las implementaciones concretas que usan.

Indirección

¿Cómo evitar un acoplamiento directo entre dos clases?

Asignar la responsabilidad a un **intermediario** que crea una indirección

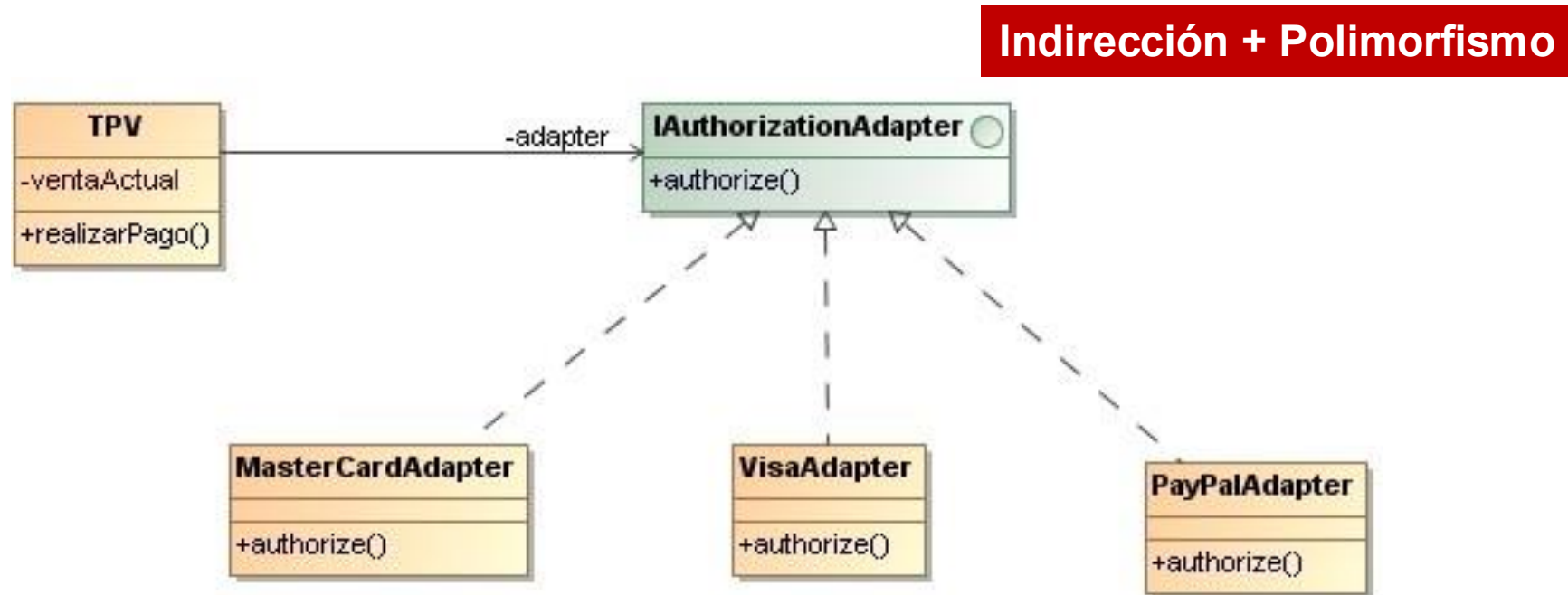


Situación que se produce en muchos patrones de diseño:
Adaptador, Observador, Proxy, ..

Variaciones Protegidas (VP)

¿Cómo diseñar clases o aplicaciones de manera que los cambios no tengan un impacto indeseable sobre ellas (protección frente al cambio)?

1. **Identificar los puntos de variación previstos** y asignar responsabilidades para crear una **interfaz** alrededor de ellos



Variaciones Protegidas (VP)

2. Principio de la Ocultación de Información (POO)
3. Acceso Uniforme (POO)
4. Principio de Sustitución de Liskov (POO)
5. Principio “no hables con extraños”
6. Principio Abierto-Cerrado (POO)
7. **Metaprogramación e Introspección** (Tema 2)

- Programar hacia la interfaz
- Favorecer la composición frente a la herencia

Ocultación de la información

- **Minimizar la accesibilidad** de las clases y sus elementos.
 - No declarar atributos públicos, sino que sean privados y la clase ofrezca métodos *para acceder y modificarlos*.
 - Ocultar detalles de implementación al código cliente.

```
public class Vehiculo {..  
    public double velocidad;  
}
```

```
public class Vehiculo {..  
    private double velocidad;  
    public double getVelocidad() {  
        return velocidad;  
    }  
    public void setVelocidad(double velocidad){  
        //posible control restricciones  
        // posible cambiar representación  
    }  
}
```

Beneficios

- Es posible establecer **restricciones sobre los valores de los atributos** que son controladas en métodos set
- Es posible **cambiar la representación** de los datos sin afectar al código cliente.
- Es posible ejecutar efectos laterales, por ejemplo, notificar el cambio a otros objetos.

Principio de Acceso Uniforme

- “Conveniencia de ofrecer los servicios de una clase mediante una notación uniforme, que no distinga si se han implementado mediante almacenamiento (atributo) o cálculo (método)”
 - Se consigue accediendo con métodos getter.
- El lenguaje puede proporcionar soporte: la misma sintaxis para invocar a un método y acceder a un atributo (por ejemplo, Eiffel, C#, Scala) .

```
cu: Cuenta; s: Real;
```

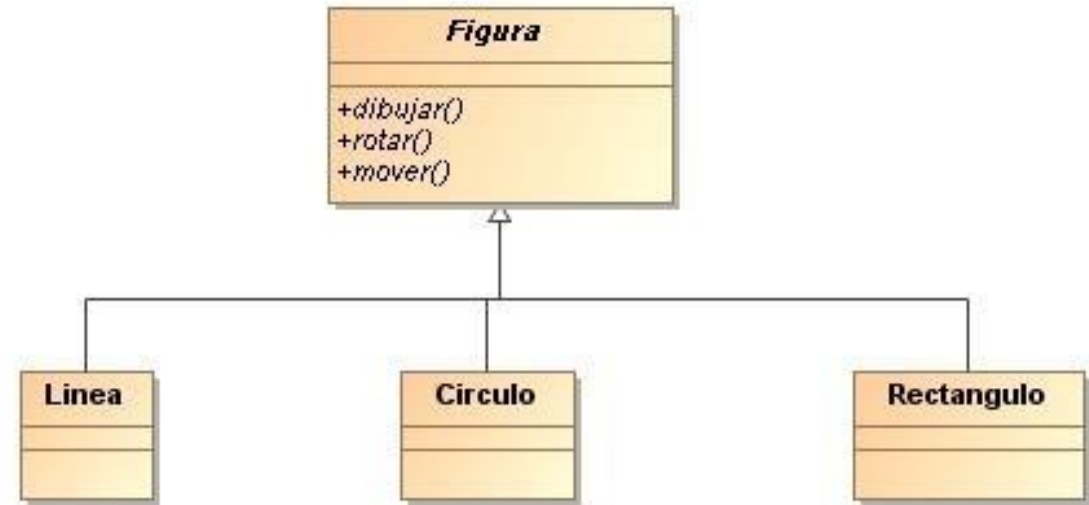
```
s := cu.saldo    // función o atributo
```

Eiffel

- Se favorece que una responsabilidad implementada como un atributo pase a ser calculada.

Principio de Sustitución de Liskov (LSP)

- Es una consecuencia de la herencia y el polimorfismo.
- Siempre que una entidad tiene como tipo A, dinámicamente podrá referenciar a instancias de cualquier subclase de A.
- Un programador puede violarlo si una subclase no es consistente con la superclase.



```
void pintar (Collection<Figura> figuras) {  
    for (Figura figura: figuras)  
        figura.dibujar();  
}
```

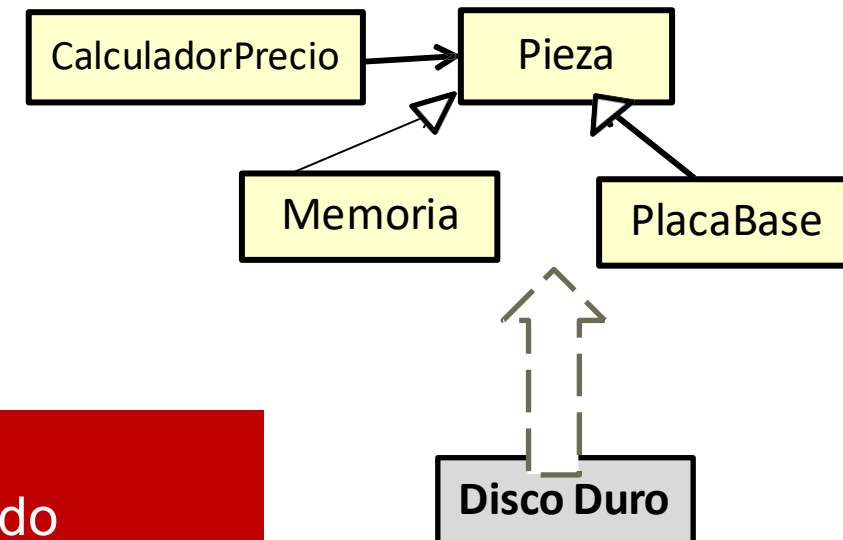
**Código
genérico**

Principio Abierto-Cerrado

- Favorece la extensibilidad
 - Añadir nuevo comportamiento sin modificar el código existente.
- Los módulos deberían ser **cerrados para la modificación** (y listos para usarse) al tiempo que **abiertos para ser extendidos**.
- Conviene tener el mayor número posible de clases abiertas y cerradas al mismo tiempo:
 - Uso de Herencia y Polimorfismo

```
public double precioTotal(Pieza[] piezas){  
    double total = 0.0;  
    for (Pieza pieza: piezas)  
        total = total + pieza.getPrecio();  
    return total;  
}
```

Satisface el
principio abierto-cerrado
(con respecto a los tipos de piezas)



¿Y si se introduce una política de descuentos que depende del tipo de Pieza?

Principio Abierto-Cerrado

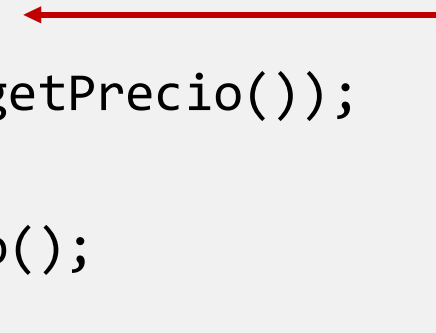
La clase CalculadorPrecio **NO satisface** el principio abierto-cerrado

```
public double precioTotal(Pieza[] piezas) {  
    double total = 0.0;  
    for (Pieza pieza: piezas) {  
        if (pieza instanceof PlacaBase)  
            total = total + (0.90 * pieza.getPrecio());  
        else if (pieza instanceof Memoria)  
            total = total + (0.75 * pieza.getPrecio());  
        else  
            total = total + pieza.getPrecio();  
    }  
    return total;  
}
```

por el cálculo del precio final



por usar los nombres de estas clases



Principio Abierto-Cerrado

```
public class Pieza {..  
    private double precio;  
    public double getPrecio() {  
        return precio; }  
}
```

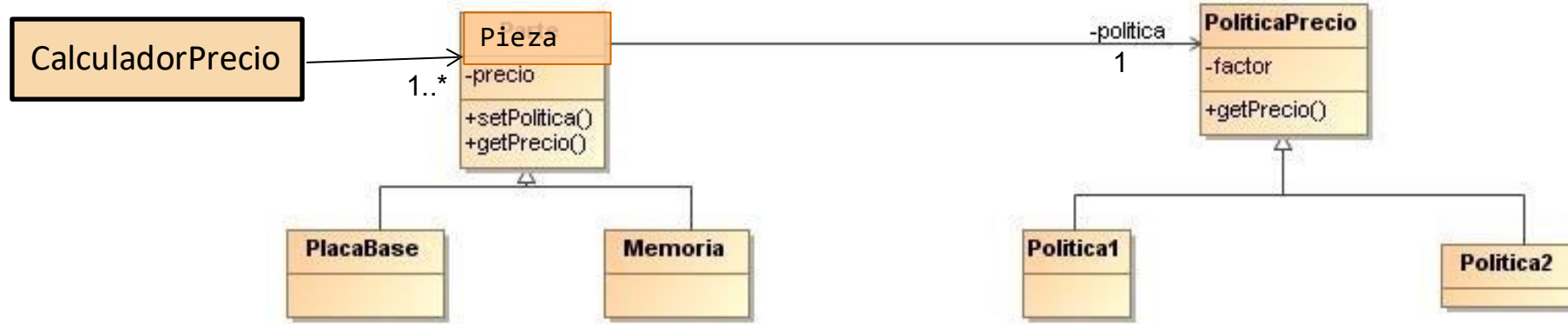
```
public class PlacaBase extends Pieza {..  
  
    public double getPrecio() {  
        // return (0.85 * precio);  
        return (0.90 * precio);  
    }  
}
```

El descuento es implementado en el método `getPrecio()`

¡Hay que cambiar estas clases si cambia el descuento!

Las clases `Pieza` **NO** satisfacen el principio abierto-cerrado (con respecto los descuentos)

Principio Abierto-Cerrado



```
public double precioTotal(Pieza[] piezas){
    double total = 0.0;
    for (Pieza pieza: piezas) {
        total = total + pieza.getPrecio();
    }
    return total;
}
```

```
abstract class Pieza {..
    double precio;
    PoliticaPrecio politica;
    public double getPrecio() {
        return politica.getPrecio(precio);
    }
}
```

CalculadorPrecio y Pieza y sus subclases **satisfacen** el Principio Abierto -Cerrado

Principio “No hables con extraños”

- No enviar mensajes sobre objetos indirectos, sino sobre la instancia actual (**self**), **parámetros** de métodos, **atributos** de la instancia actual y **elementos de colecciones** de la instancia actual. Evitar recorrer largos caminos en la estructura de objetos: “diseño frágil”

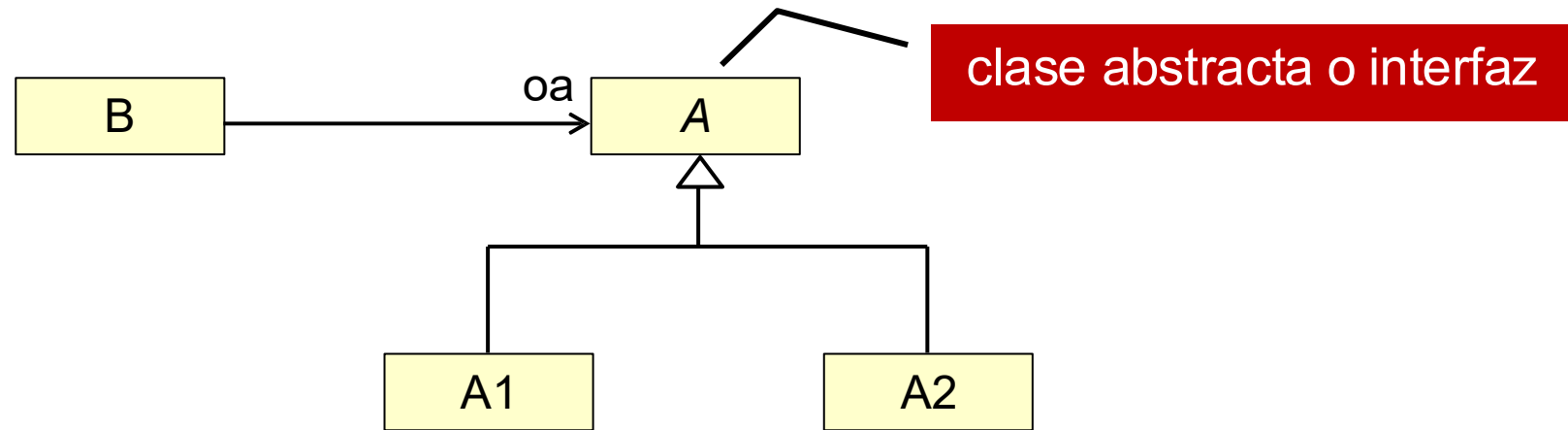


```
class TPV {
    private Venta venta;
    public void metodoFragil () {
        Dinero cantidad =
            venta.getPago().getCantidadEntregada()
        ...
    }
}
```

Dinero cantidad =
`venta.getCantidadEntregadaEnPago()`

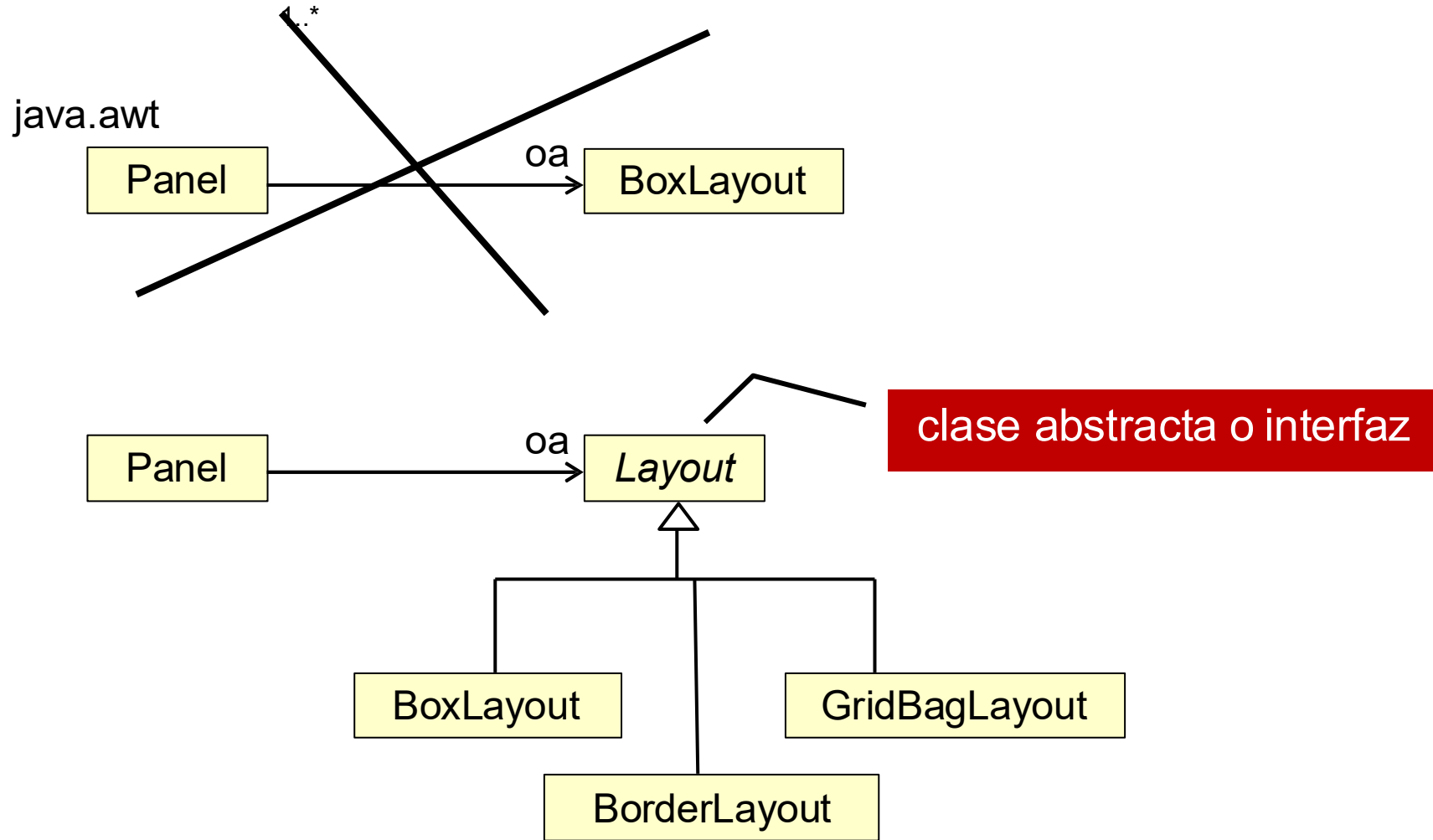
Programar hacia la interfaz

- ***“Programar hacia la interfaz, no hacia la implementación”***
 - No declarar variables de clases concretas sino abstractas.
 - Patrones de creación permiten que un sistema esté basado en términos de interfaces y no en implementaciones.

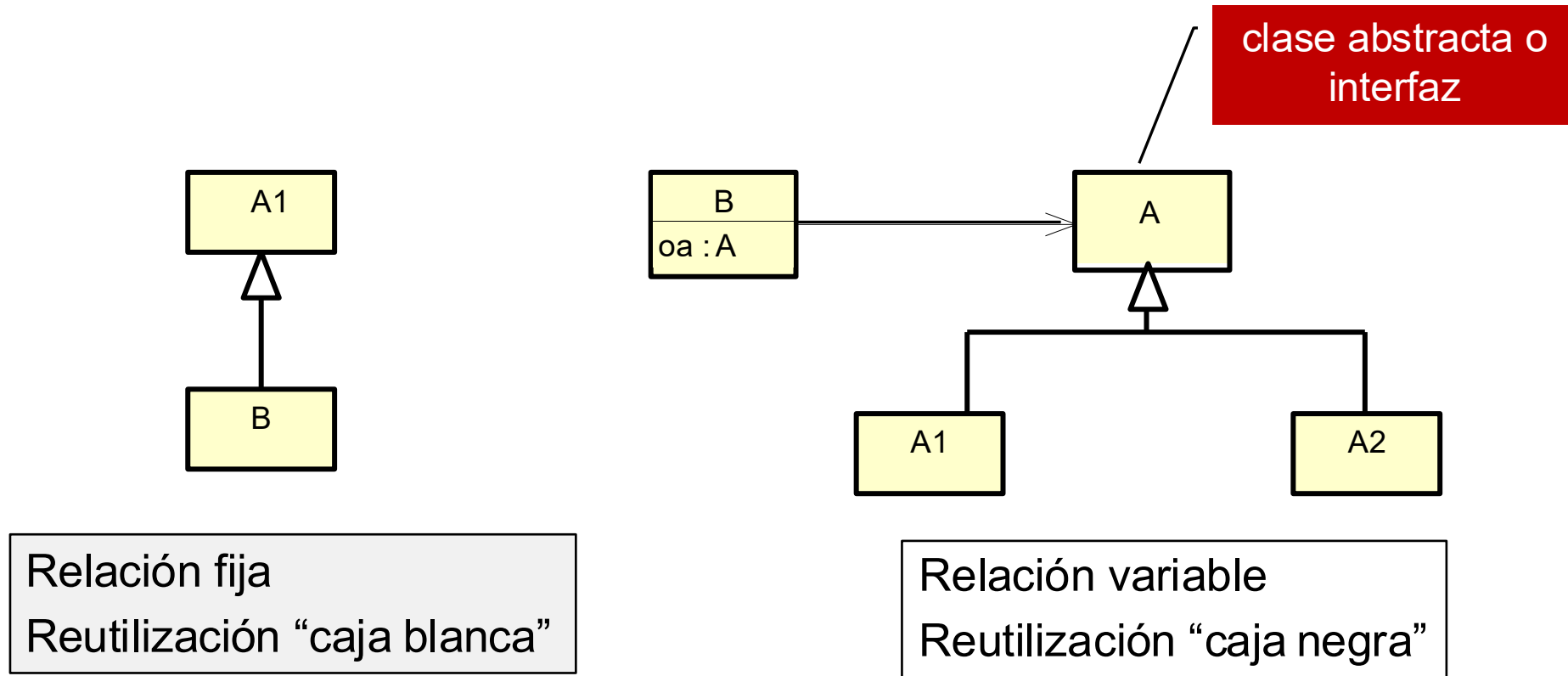


Una instancia de B puede estar ligada a una instancia de cualquier subclase de A.

Programar hacia la interfaz



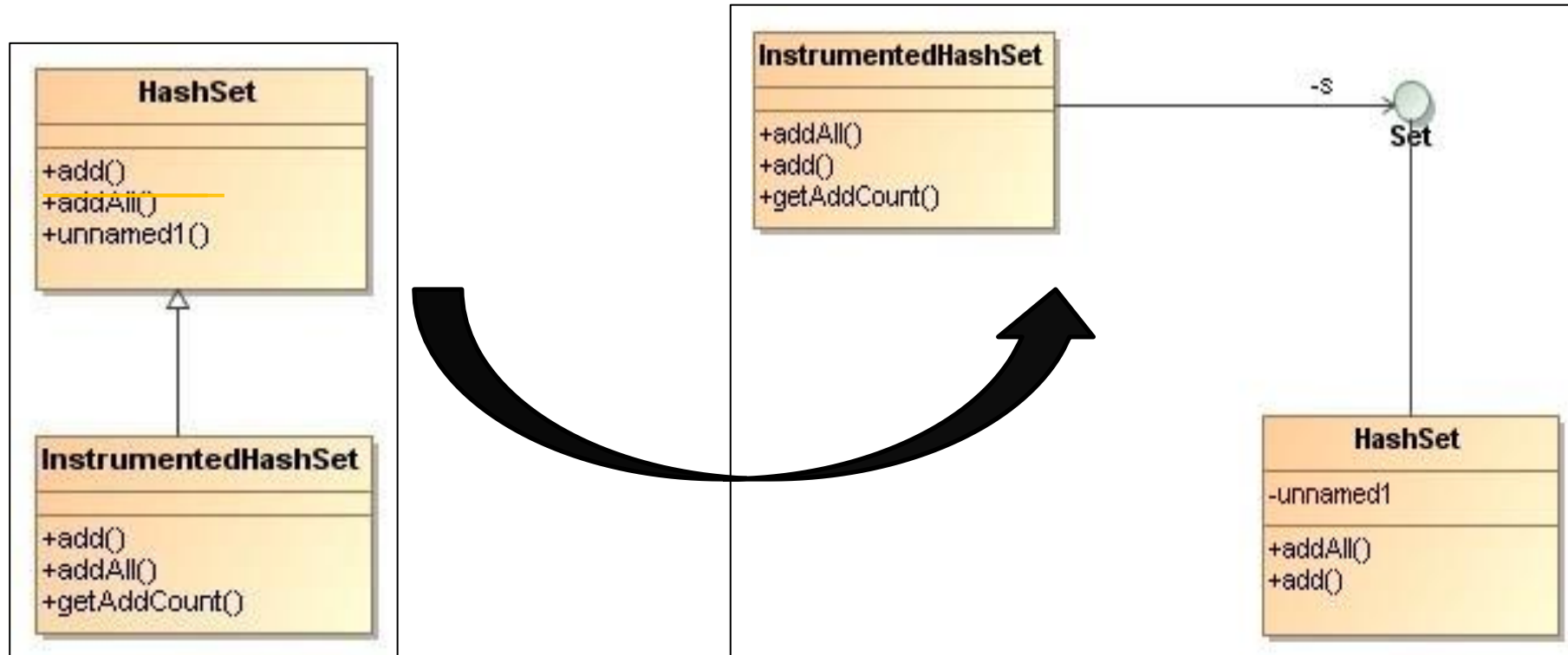
Favorecer composición frente a herencia



Composición y herencia trabajan juntas

Favorecer composición frente a herencia

(“Effective Java”, Joshua Bloch, item 16)



Relación fija
Reutilización “caja blanca”

Relación variable
Reutilización “caja negra”

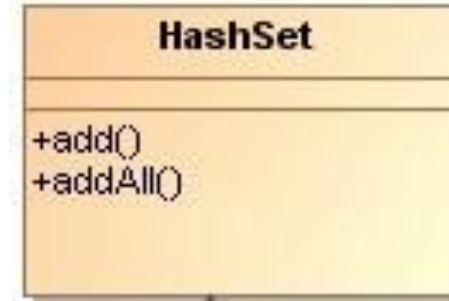
**¿Por qué se debe favorecer la
composición frente a la herencia?**



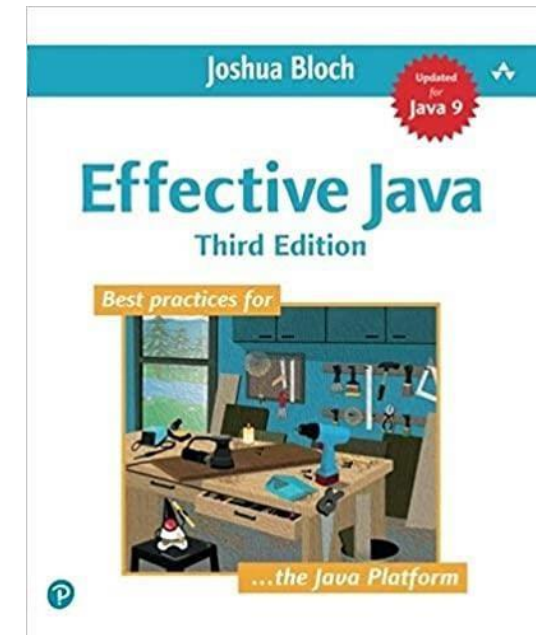
QUINTO

Ejemplo de “Effective Java”, Joshua Bloch (item 16)

Supongamos una clase `HashSet` que implementa los métodos `add()` para añadir un elemento y `addAll()` para añadir una colección de elementos.



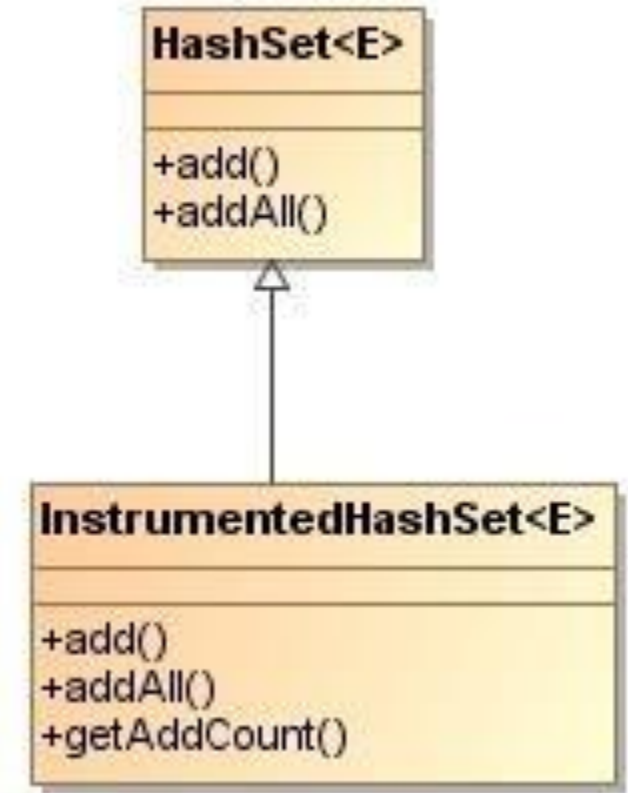
Deseamos aprovechar `HashSet` para crear una clase `InstrumentedHashSet` que nos permita llevar cuenta del número de elementos que hemos insertado en el conjunto, no los que tiene en el momento actual.



Ejemplo de “Effective Java”, Joshua Bloch(item 16)

```
public class InstrumentedHashSet<E> extends HashSet<E>{
    // Contador número de elementos añadidos
    private int addCount = 0;

    public InstrumentedHashSet(Collection c){
        super(c);
    }
    public InstrumentedHashSet(int initCap) {
        super(initCap);
    }
    public boolean add(E e) {
        addCount++;
        return super.add(e);
    }
    public boolean addAll(Collection<? Extended E> c) {
        addCount += c.size();
        return super.addAll(c);
    }
    public int getAddCount() {
        return addCount;
    }
}
```



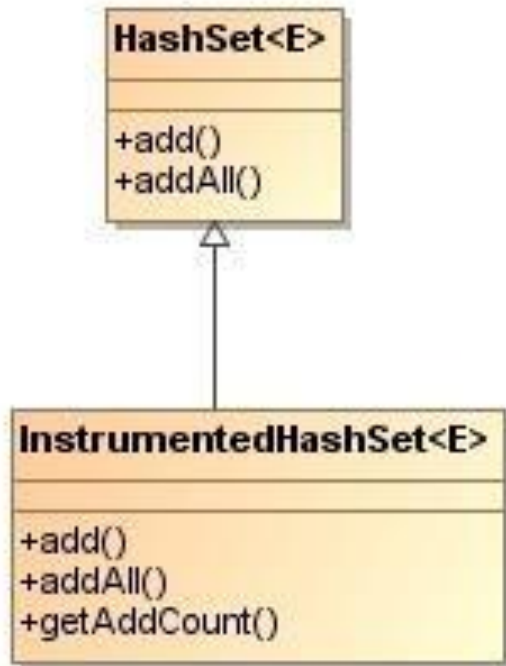
No tiene el
comportamiento
esperado

Ejemplo de “Effective Java”, Joshua Bloch (item 16)

```
InstrumentedHashSet s= new InstrumentedHashSet<String> ();  
  
s.addAll (Arrays.asList ("uno", "dos", "tres")) ;
```

¡¡ s.getAddCount() debería retornar 3, pero retorna 6 !!

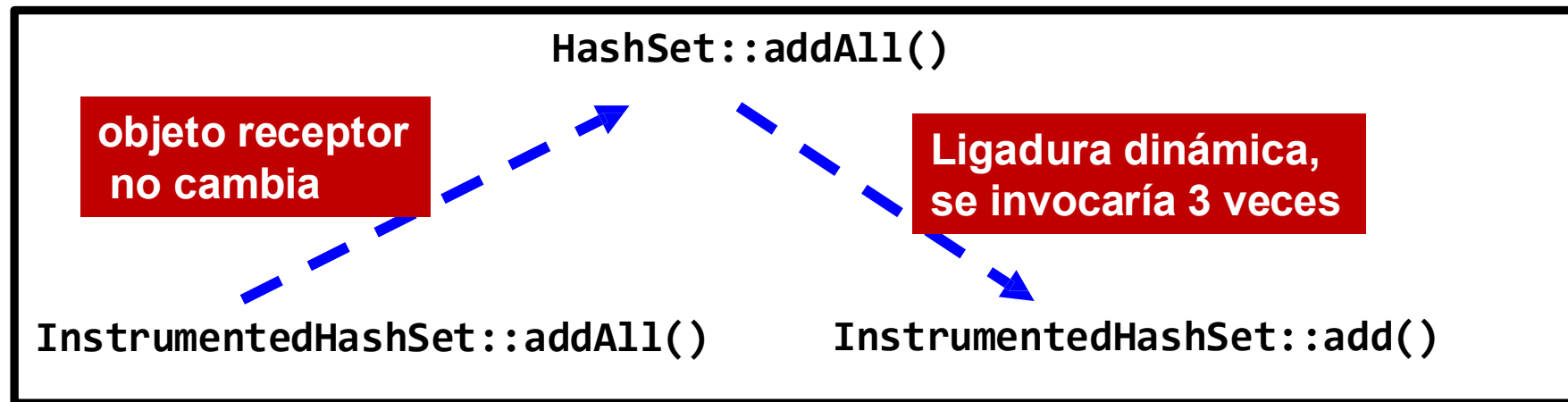
¿Por qué?



```
public boolean addAll(Collection<E> c) {
    for (E e:c) this.add(e);
}
```

```
public boolean addAll(Collection<E> c) {
    addCount += c.size();
    return super.addAll(c);
}
public boolean add(E e) {
    addCount++;
    return super.add(e);
}
```

```
InstrumentedHashSet s=
    new InstrumentedHashset<String>();
s.addAll(Arrays.asList("uno","dos","tres"));
```



Ejemplo de “Effective Java”, Joshua Bloch (item 16)

Problema

- El método `addAll(Collection c)` de la superclase, lo que hace es recorrer la colección `c` y para cada uno de los elementos llama al método `add()`.

Solución

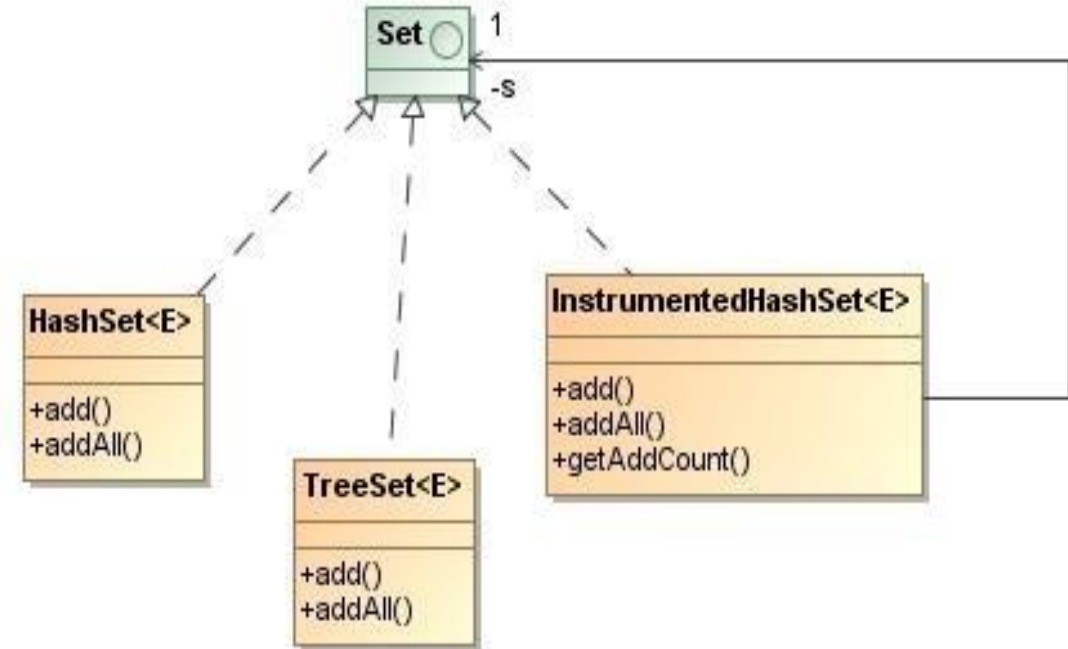
- Las dos clases implementan un interfaz con las operaciones `add()` y `addAll()`.
- `InstrumentedHashSet`, en vez de heredar, referencia a un `HashSet` en quien delega las operaciones, de esta manera se limitará a llevar un contador.
- Las operaciones que no necesiten ser modificadas se implementarán por delegación.

Ejemplo (“Effective Java”, Joshua Bloch)

```
public class InstrumentedSet<E>
    implements Set<E>{

    private final Set<E> s;
    private int addCount = 0;
    public InstrumentedSet(Set s)
        { this.s = s; }
    public boolean add(Object o) {
        addCount++;
        return s.add(o);
    }
    public boolean addAll(Collection c) {
        addCount += c.size();
        return s.addAll(c);
    }
    public int getAddCount() {
        return addCount;
    }

    // resto de métodos de Set<E> que delegan en el método de igual
    // nombre en Set<E>
    public void clear() { s.clear(); }
    public boolean contains(Object o) { return s.contains(o); }
    ...
}
```



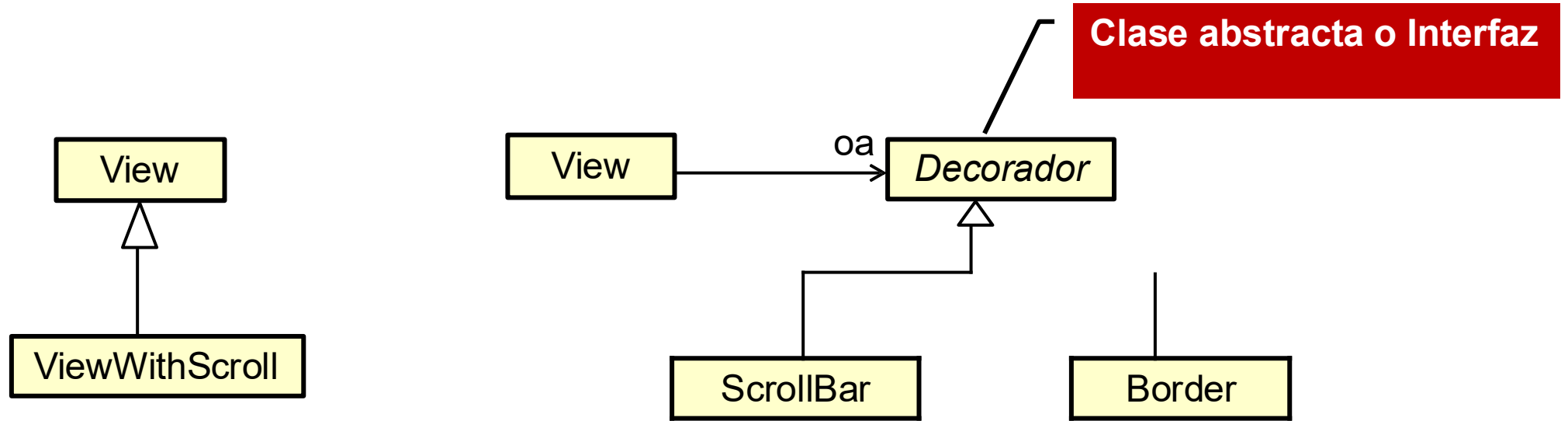


¡¡La herencia viola la encapsulación !!



- Una **subclase depende** de los detalles de implementación de su **superclase** para funcionar correctamente:
 - **Más difícil de comprender**
 - **Si la implementación de la superclase cambia**, podría afectar al **comportamiento de la subclase** aunque no se haya tocado el código de esta.

Favorecer composición frente a herencia



Relación fija
Reutilización "caja blanca"

Relación variable
Reutilización "caja negra"

Single
responsibility



Open-Closed
Principle



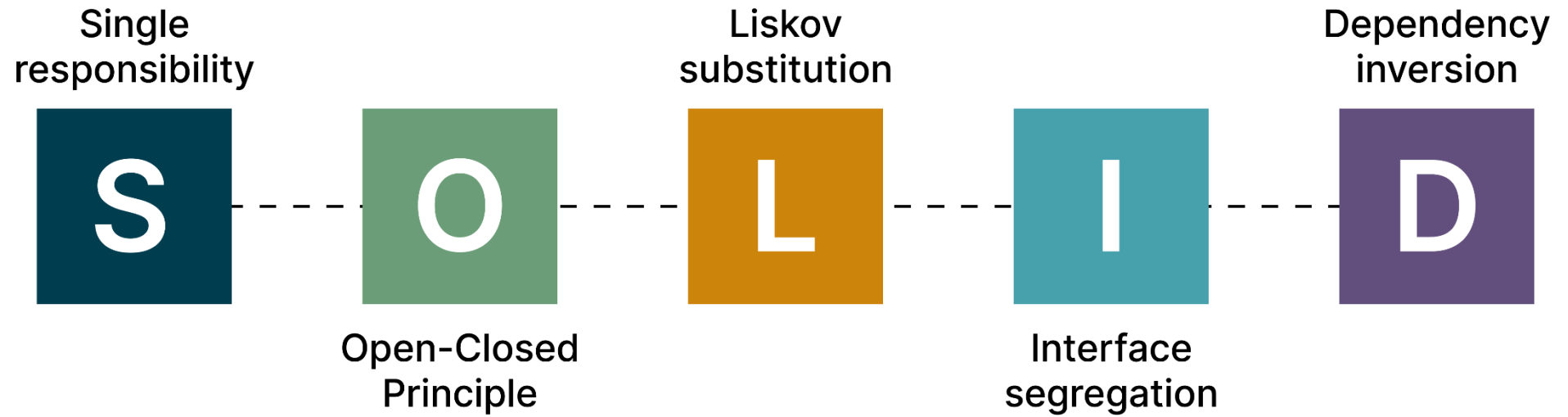
Liskov
substitution



Interface
segregation



Dependency
inversion



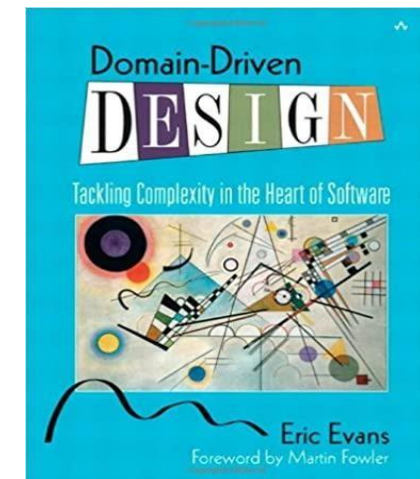
1. **Principio de Responsabilidad Única (Single Responsibility Principle, SRP):**
 - Este principio establece que una clase debería tener solo una razón para cambiar. En otras palabras, una clase debería tener solo una responsabilidad. Esto ayuda a mantener el diseño del sistema limpio y modular.
2. **Principio Abierto/Cerrado (Open/Closed Principle, OCP):**
 - Según este principio, las entidades de software (clases, módulos, funciones, etc.) deberían estar abiertas para la extensión, pero cerradas para la modificación. Esto implica que deberías poder agregar nuevas funcionalidades extendiendo el sistema, no modificando el código existente.
3. **Principio de Sustitución de Liskov (Liskov Substitution Principle, LSP):**
 - Este principio establece que los objetos de una clase derivada deberían poder reemplazar a los objetos de la clase base sin afectar la corrección del programa. Es fundamental para garantizar que las clases derivadas mantengan el comportamiento definido en las clases base.
4. **Principio de Segregación de Interfaces (Interface Segregation Principle, ISP):**
 - Según este principio, es mejor tener muchas interfaces específicas, en lugar de una interfaz única general. Esto ayuda a mantener los sistemas desacoplados y a adherirse al principio de responsabilidad única.
5. **Principio de Inversión de Dependencias (Dependency Inversion Principle, DIP):**
 - Este principio establece que los módulos de alto nivel no deberían depender de los módulos de bajo nivel, ambos deberían depender de abstracciones. Además, las abstracciones no deberían depender de los detalles; los detalles deberían depender de las abstracciones. Esto promueve un acoplamiento débil y una estructura de sistema más modular.

Siguiendo estos principios, los desarrolladores pueden crear sistemas más organizados, flexibles, y mantenibles, lo que a largo plazo facilita la adición de nuevas funcionalidades y la resolución de bugs.

En la mayoría de los dominios, la calidad y productividad depende de **buenas prácticas, técnicas y herramientas** (“la pila de herramientas”).

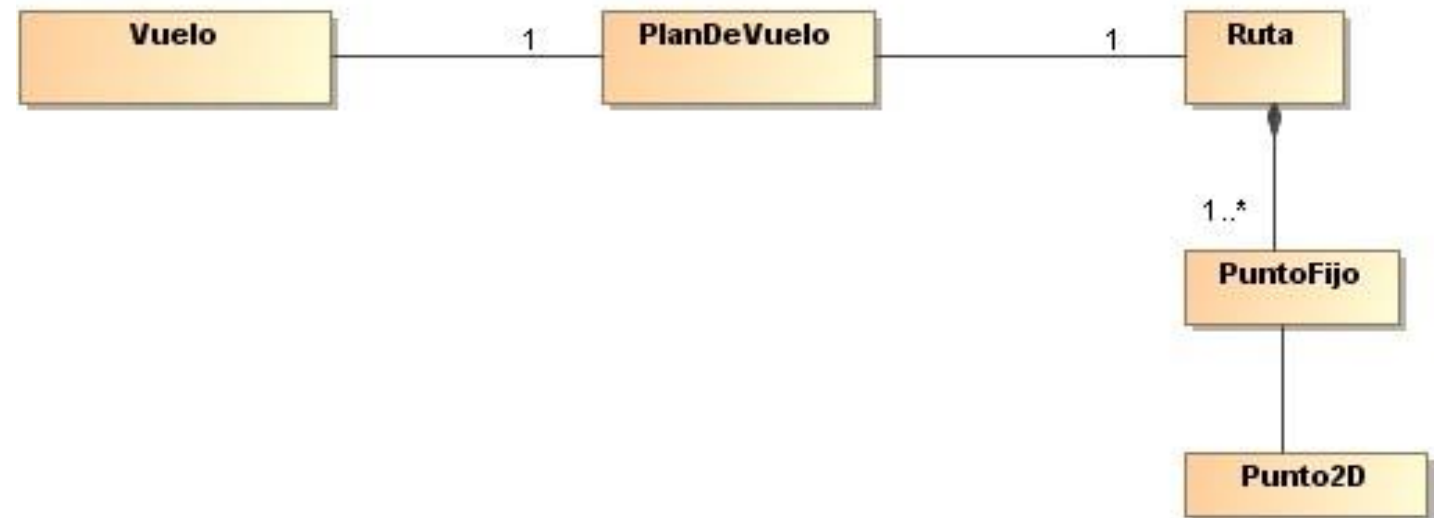
Técnicas de desarrollo

- **Patrones de diseño**
 - Forma de reutilizar la experiencia de buenos programadores
 - Soluciones a problemas de diseño recurrente
- **Desarrollo basado en componentes**
 - Construcción basada en componentes que se conectan de acuerdo a una arquitectura dada.
- **eXtreme Programming**
 - Conjunto de buenas prácticas
 - Integración continua
 - Testing y refactoring continuo --> Desarrollo dirigido por pruebas (TDD)
- **Diseño dirigido por el dominio (DDD)**
 - Importancia de crear un modelo del dominio, el cual proporciona un vocabulario y guía el diseño y la implementación.
 - **Será utilizado en el desarrollo del caso práctico**



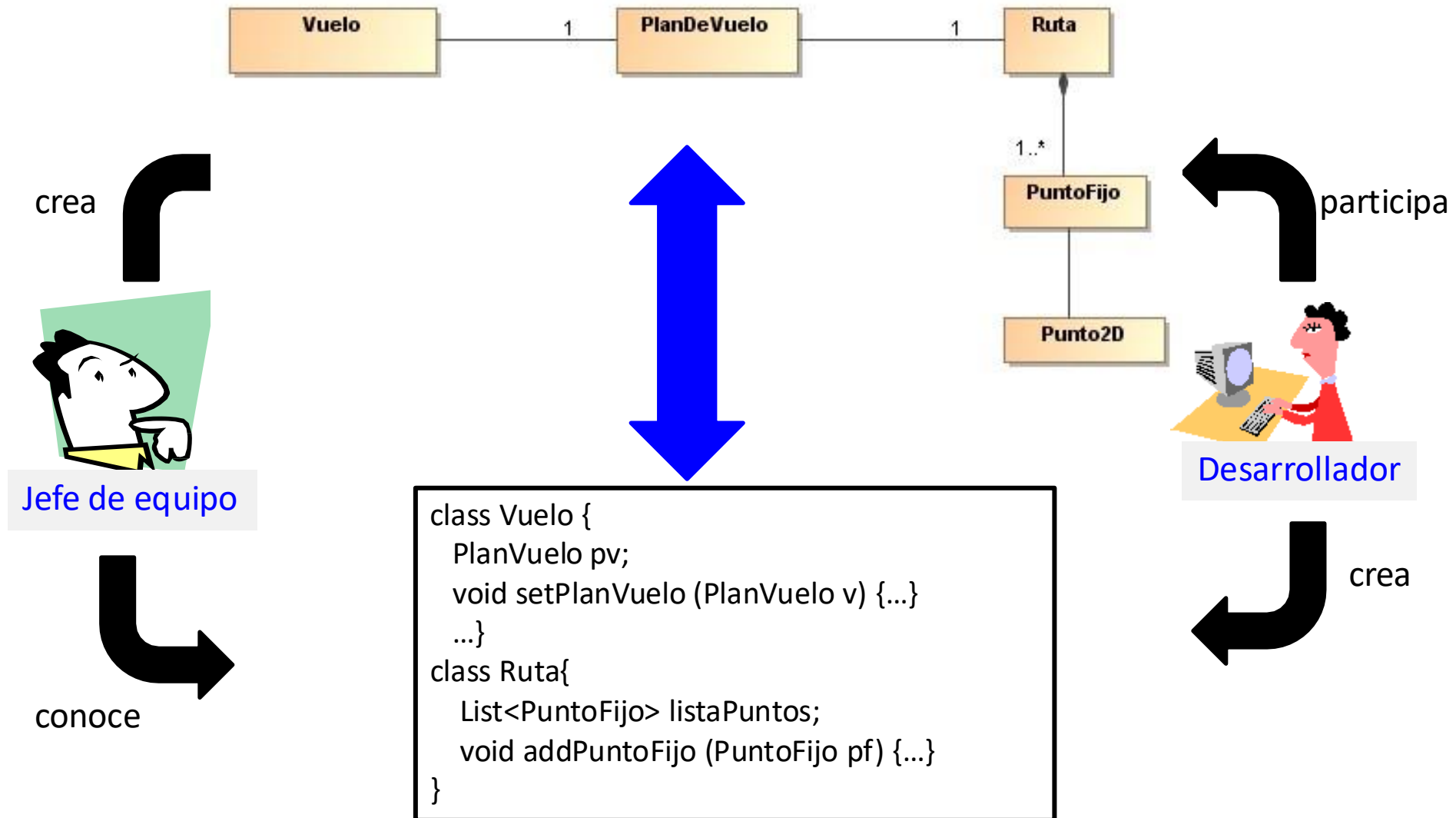
Diseño Dirigido por el Dominio (DDD)

- Un **modelo del dominio** dirige el desarrollo.
 - Jefe de equipo representa el *conocimiento* del dominio en colaboración con el experto del dominio (ideal).
 - Representación con gráficos (p.e., diagramas de clases UML) y/o texto
- Crean un **lenguaje común compartido (lenguaje ubicuo)**
- El *modelo* es orientado a la creación de código
 - Entidades del dominio son clases
 - Relaciones entre entidades son asociaciones

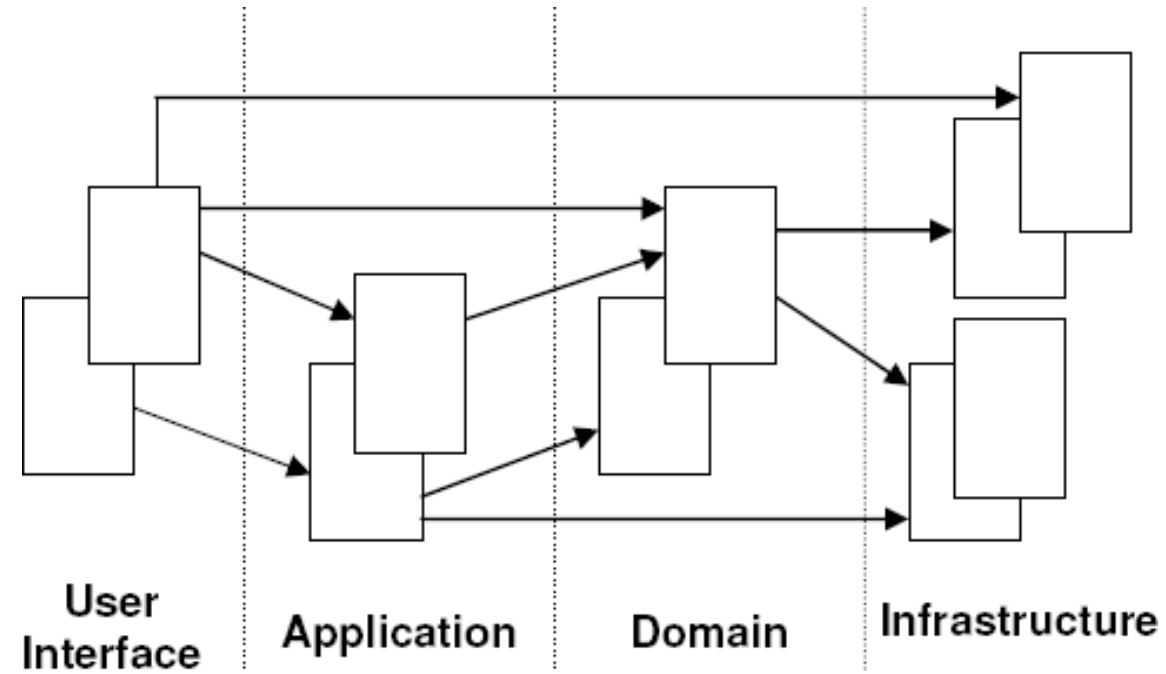


* En esta asignatura se dará una primera aproximación breve y sesgada del concepto de DDD discutido en el libro de Evans. En la asignatura de software continuación del segundo cuatrimestre se profundizará y completará su estudio.

DDD. Modelo del Dominio



DDD. Arquitectura en capas



DDD. Tipos de Objetos

- **Entidad**

- Objetos que **tienen una identidad** que se mantiene a lo largo de la vida del sistema sin importar que cambios tengan lugar. **Ejemplos:** Usuario, Cuenta, Mensaje, Viaje, y Hospital.

- **Value Object**

- Objetos que **no tienen una identidad**. Pueden ser compartidos y deberían ser inmutables. **Ejemplos:** Fecha, Punto, Complejo, Ruta,...

- **Servicios de Dominio**

- Objetos que denotan “**algún proceso o transformación en el dominio** que no es una responsabilidad natural de una entidad o value object”, y no parece adecuado asignarlo a una de ellas. **No tienen estado**. Ejemplos: Calculador de descuentos, Generador de números de factura, generador de código de pedido, ...

- **Módulo**

- Organizar el modelo en varios módulos para dominar la complejidad. Favorece la cohesión. Usar una interfaz (fachada) para reducir el acoplamiento --> **Controlador**

DDD. Control ciclo de vida de objetos

- **Agregados**

- **Grupo de objetos relacionados** que son considerados como una unidad.
- El acceso a través de un **objeto raíz que es de tipo entidad**.
- El objeto raíz controla que los objetos que agrega satisfacen el invariante tras un cambio.
- Se aplica patrón Creador: el objeto raíz crea a los objetos que agrega



* En el libro de Evans se hace explícita la necesidad del uso de relaciones entre agregados basados en IDs y la importancia del objeto raíz para la gestión de los objetos agregados.

DDD. Control ciclo de vida de objetos

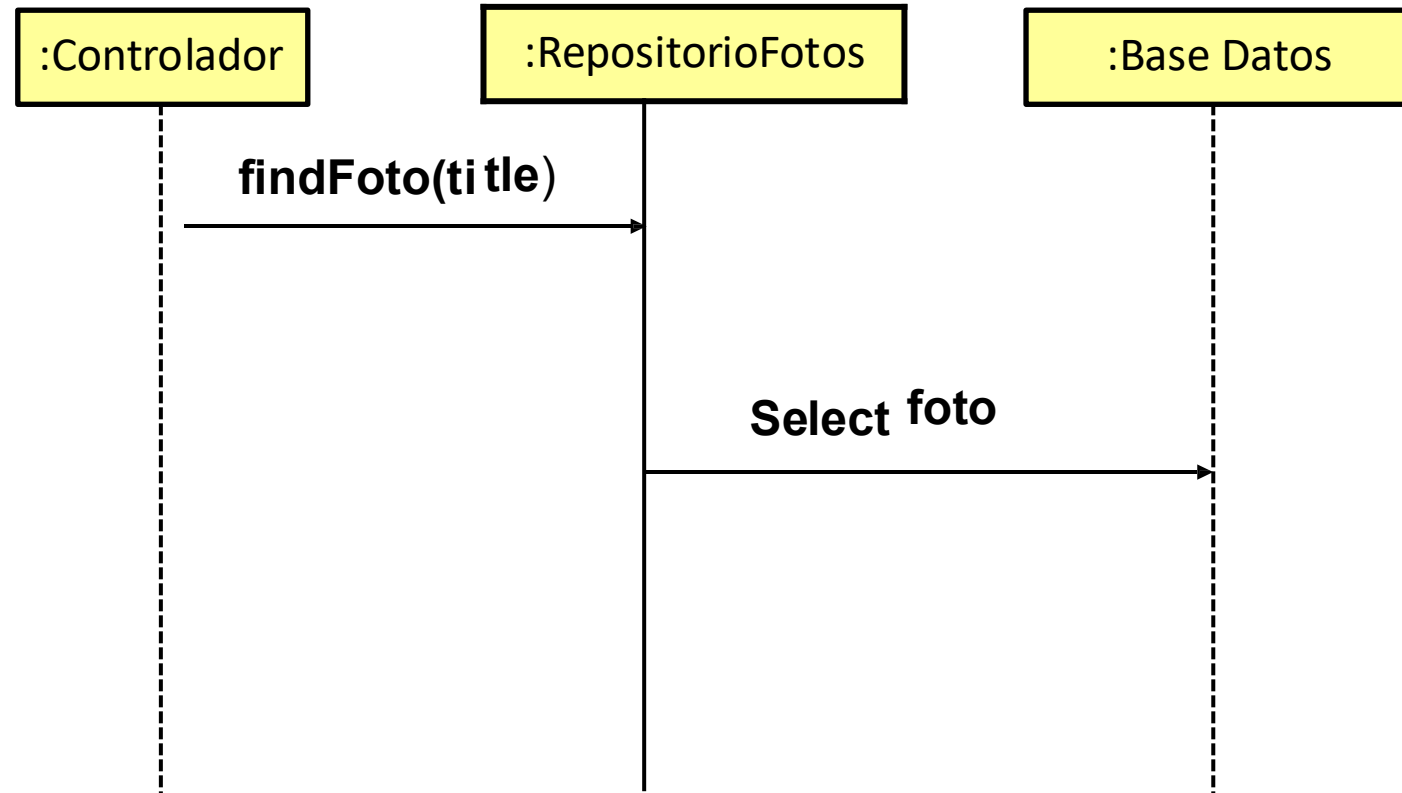
- **Factorías**

- Clases encargadas de la construcción de objetos. Usar en vez de constructores para una creación sin especificar el tipo concreto o para crear un objeto complejo, por ejemplo, agregados o lógica de creación compleja
- Patrones de diseño GoF: *Factoría Abstracta*, *Método Factoría* y *Builder*.

- **Repositorios (Catálogos)**

- Encapsulan la lógica necesaria para **recuperar objetos** de un tipo de Entidad (por ejemplo, Cliente, Vuelo,...)
- Lugar de almacenamiento para objetos que es accesible globalmente: patrón *Singleton*.
- Métodos para **añadir/eliminar y para buscar** objetos a través de un identificador, uno o más atributos, o un criterio de selección.
- No son responsables de crear objetos.

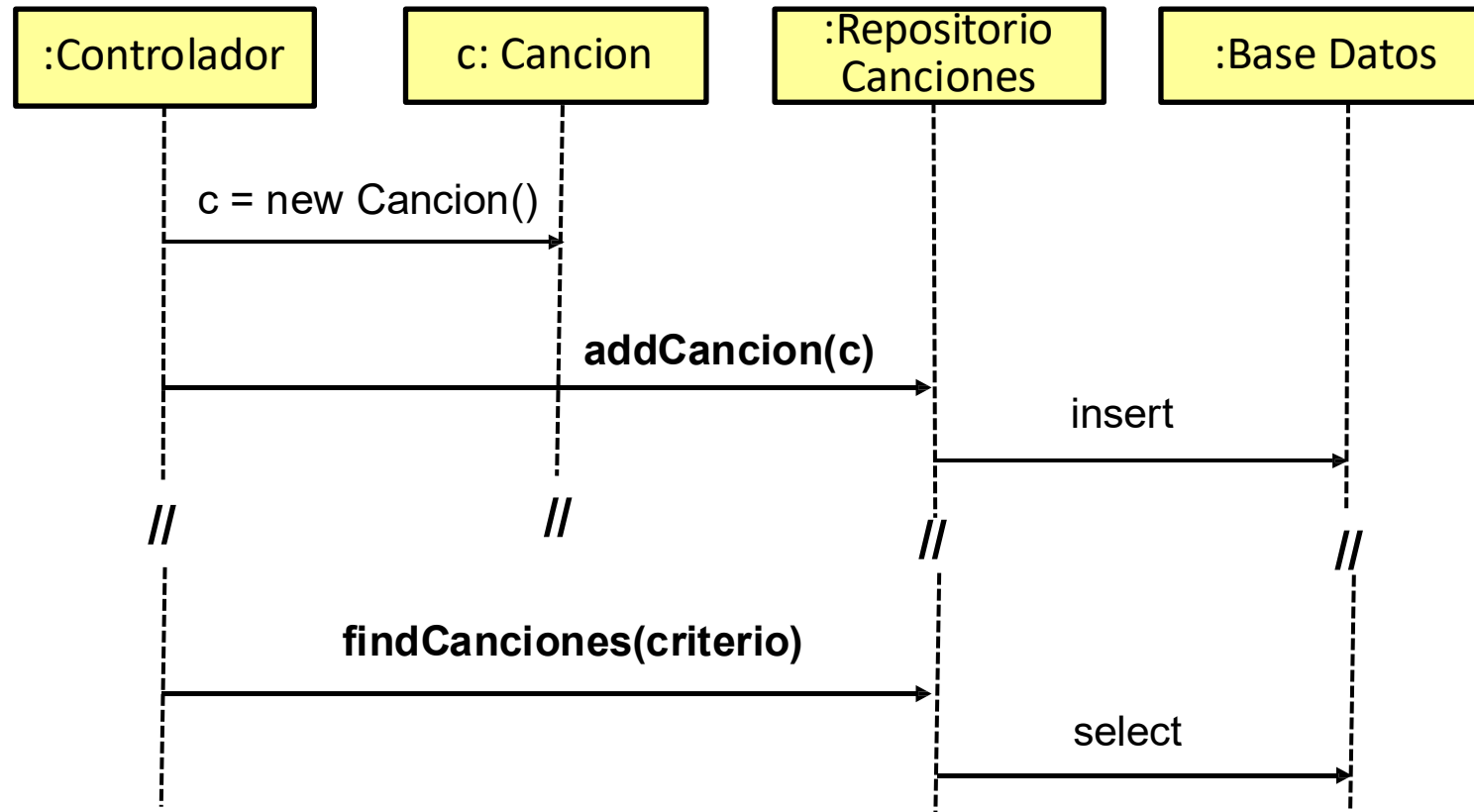
DDD. Repositorio



Aproximación 1. En nuestro caso, los repositorios tendrán todos los objetos almacenados de la entidad y se cargarán al arrancar la aplicación.

Aproximación 2. ¡¡ Se podrían (deberían) **cargar bajo demanda !!**

DDD. Repositorio



- Repositorios **no crean objetos**
- Incluyen métodos **add, remove y find**